
ICT UNIVERSITY

Faculty of Information and Communication Technologies

Software Design Document

Multi-tenant University ERP

Academic • Marketing & Finance
Administration & Human Resources

Document identifier	ERP-SDD-001
Version / status	1.1 / Group review draft
Baseline	Backend 6e10c54; frontend 6cc6859; reviewed 13 September 2026
Course	SEN4121 – Large System Environment, Summer 2026
Instructor	Engr. Tanwi Nkiamboh
Scope	React frontend and four-service backend, with deployment and integration boundaries
Prepared by	Four-member Software Engineering project group

Group members

No.	Full name	Student / registration ID
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____

Reading guide

Eight-page architecture narrative, followed by detailed design decisions, 21 PlantUML views, frontend/API integration tables and an automatically generated catalogue of 97 service-local routes and 42 persisted models.

This document describes observed code, specified requirements and unverified targets separately. It is not a certification of production readiness, statutory payroll compliance or successful execution of application tests.

Document control

Version	Date	Change
1.0	13 September 2026	Initial code-grounded specification with editable PlantUML sources and local PDF build.
1.1	13 September 2026	Assignment alignment, four-member submission forms and clarified UML execution/relationship notation.

Review and approval

Responsibility	Name / signature	Date
Product / academic representative	_____	_____
Finance and payroll representative	_____	_____
Technical lead / document reviewer	_____	_____
Course submission approval	_____	_____

Authority and interpretation

The supplied four-page SEN4121 project brief is the source of stakeholder obligations. Source code, migrations and tests determine what is implemented. Existing operational reports are historical evidence, not rerun results. The ISO/IEC/IEEE 29148 reference is used as an organizational alignment for requirements; no conformance assessment or reproduction of that standard is claimed.

Group submission responsibilities

All four members must review and understand both documents. Complete the individual contribution statements, obtain all four signatures on each, and sign the submission declaration supplied in the technical-report appendices. The report's source acknowledgments form part of the submission and defense. Blank names, signatures and evidence fields must be completed by the group; this review draft records no approval or individual contribution.

Pagination policy

Read the SRS for stakeholder requirements and acceptance criteria. This SDD describes both the React frontend and the backend implementation. Main narrative pages are deliberately bounded; extensive requirements, interface inventories and diagrams are appendices. The brief limits the combined SRS and technical report to 20 pages excluding appendices and diagrams. The package budgets six SRS narrative pages, eight SDD narrative pages and three front-matter pages per document: 20 combined non-appendix pages including covers, control pages and contents. Appendices are explicitly labeled and excluded from that total; this is not two separate 20-page allowances.

Contents

Document control	i
1 Architectural baseline and system decomposition	1
2 Frontend architecture and interaction design	2
3 Identity, authorization and trust boundaries	3
4 Academic domain and asynchronous billing design	4
5 Finance, payments and analytical projections	5
6 HR workflows and payroll computation	6
7 Deployment, configuration and delivery pipeline	7
8 Operational qualities, failure handling and evolution	8
A Design decisions, consistency and data rationale	9
B Detailed frontend integration and UX coverage	11
C Message, API and numerical contracts	13
D UML and architecture diagram suite	15
E Generated service-local route catalogue	38
F Generated persisted-model dictionary	42
G Setup, validation and operational handover	54

1 Architectural baseline and system decomposition

1.1 Scope and viewpoint

This document describes frontend baseline 6cc6859 in `ERP_System` and backend baseline 6e10c54 in `ERP_System_backend`. All paths are backend-relative unless prefixed `../ERP_System`. The supplied SEN4121 brief drives mandatory boundaries; the SRS identifies requirements and implementation gaps. Figures distinguish code-backed structures from conceptual scaling and recovery targets.

Boundary	Responsibility	Technology / owned data
Web frontend	Session UX, route shell, student academics/payments, staff self-service	React 19, TypeScript, Vite; no private database.
Identity	Users, tenants/campuses, Argon2 login, RS256 signing, refresh families	FastAPI; <code>identity_db</code> .
Academic	Enrollment, curriculum, grades/appeals, attendance, exams/reports	FastAPI + outbox worker; <code>academic_db</code> .
Finance	Fees, invoices, payments/ledger, expenses, campaign ROI and summaries	FastAPI + combined worker; <code>finance_db</code> .
HR	Recruitment, employee records, leave, shifts, payroll, assets	FastAPI; <code>hr_db</code> .
Gateway	Single-origin entry, proxy routes, delegated authentication, IP budgets	Nginx; no business persistence.

1.2 Structural decisions

Bounded contexts own data and deployment images. APIs do not perform cross-database joins. Academic acceptance triggers Finance asynchronously, avoiding a synchronous availability dependency at enrollment time. HTTP serves interactive reads and commands; the broker provides durable workflow transfer, not arbitrary service discovery. Compose DNS names and explicit Nginx upstreams constitute the static service registry.

Four databases share one PostgreSQL 17 process: this is logical ownership separation, not physical high availability. RabbitMQ and the host are additional shared failure domains. Workers reuse their service image but are separately runnable processes. There is no shared runtime business-model package; the `contracts` directory shares schemas rather than Python entities.

1.3 Trace to requirements

Implements the architectural shape of DATA-01, INT-01–03 and OPS-01. Figure 3 shows ownership and broker topology; Figure 17 shows actual containers and production prerequisites. Complete domain administrative UI, full CRUD, scheduled off-host backups and verified production rollout are not implied by these architectural components.

2 Frontend architecture and interaction design

2.1 Composition and state

`main.tsx` creates one TanStack Query client, with one query retry and no refetch-on-window-focus. It wraps `App`, `AuthProvider` and a React Router browser router. `ProtectedRoute` waits for session restoration and redirects unauthenticated users to login while preserving their pathname. It does not enforce module roles; feature pages gate their rendered content.

Route	Visible experience	Current implementation boundary
<code>/login</code>	Public login form	Zod email/password validation, inline server error, return navigation.
<code>/</code>	Welcome and role	Not a dynamic metrics dashboard.
<code>/academic</code>	Student academics	Enrollment, offering registration, grades/appeals, risk and PDFs; non-student placeholder.
<code>/finance</code>	Student invoices	Payment intent polling and receipts; non-student placeholder.
<code>/people</code>	Staff workspace	Leave, code-based attendance, payslips and notifications; all non-staff roles receive placeholder.
<code>/status</code>	Authenticated status page	Real gateway/service liveness requests, not dependency readiness.
<code>/settings</code>	Placeholder	No implemented institution/user administration UI.

2.2 Transport and presentation

Domain service adapters map snake-case HTTP fields into camel-case TypeScript models. `authFetch` attaches the in-memory bearer token and includes cookies; on 401 it uses a shared in-flight refresh promise and retries the original request once. `authFetchJson` parses error bodies into `ApiError`; generic typing is a compile-time assertion, not runtime schema validation. Report adapters fetch binary blobs and trigger downloads with temporary object URLs.

Forms use React Hook Form/Zod for login and enrollment, and controlled state in payment/leave/check-in forms. Shared Button, FormField, Skeleton, EmptyState and ErrorState components provide consistent feedback. Tailwind styling, local font assets and reduced-motion-aware transitions support a responsive dark interface. The shell includes skip navigation, desktop sidebar and mobile menu.

2.3 Important limits

Query keys such as `my-invoices` and `my-payslips` do not include principal identity; authentication transitions do not clear the shared cache. This requires explicit account-switch isolation work, not a claim of already completed client isolation. Selected download, nested-grade and payment-poll failure states are not fully surfaced. These are documented gaps, not code changes in this deliverable. Figure 4 and Appendix B detail the integration.

3 Identity, authorization and trust boundaries

3.1 Session lifecycle

Identity signs ten-minute RS256 JWTs; resource services verify issuer, audience, signature and expiry using the public key. Only Identity mounts the private key. Refresh sessions use a SHA-256 hash of an opaque token, default fourteen-day lifetime, family ID and revocation timestamp. Rotation inserts a successor rather than overwriting the old row, enabling sequential token-reuse detection and family revocation. Login tracks failures and locks after five failures for fifteen minutes.

Frontend startup performs refresh then `/auth/me`; the resulting principal lives in React context. Access tokens are held only in a module variable, not persistent browser storage. Resource 401 responses trigger refresh/retry and an expiry callback. There is no proactive timer that forces logout at the exact JWT expiry instant. Logout revokes the presented refresh token; access tokens remain valid until expiry. Single-runtime refresh deduplication does not serialize refreshes across browser tabs.

3.2 Gateway boundary

Nginx public exceptions include health/readiness and login/refresh/logout. Protected routes use internal `/internal/verify` authentication subrequests; identity-derived headers replace client-supplied tenant/user headers. Business services still validate the bearer token independently. Default request budgets are 20/s with burst 40 and login 5/min with burst 5, per source IP. JSON APIs use the same browser origin; no separate browser-to-service CORS channel is required.

3.3 Persistence boundary

Per-request dependencies set `app.current_institution_id` and `app.is_platform_admin` using transaction-local settings. UUID values are validated before the PostgreSQL SET statement. Forced RLS applies to scoped business tables and Identity users/refresh sessions, including service-owned tables. Identity institution/campus metadata is not RLS-enabled. Bootstrap grants database ownership to the service role and revokes public connection access. A separate migration owner and non-owner runtime role are not implemented by the current bootstrap. Outbox/inbox internal tables intentionally do not follow ordinary tenant RLS. Identity credential resolution and trusted workers use explicit platform context.

3.4 Deployment qualifications

Base Compose sets development mode and disables the Secure refresh-cookie flag. The production override replaces images/key mounts but does not remove those inherited values. Host TLS and effective environment inspection are mandatory release gates. The locally public payment callback also remains gateway-protected. These configuration facts supersede optimistic comments in older documentation. AUTH-01-08 and INT-02 map to Figures 11 and 20.

4 Academic domain and asynchronous billing design

4.1 Enrollment and curriculum

An enrollment links a student UUID with program, term and campus context. Only accepted enrollment status exists; there is no submitted/admissions approval state machine. Student/program/term uniqueness prevents duplicate records. The HTTP transaction inserts both enrollment and outbox event, then commits without calling Finance. Student creation is self-only; staff/admin supplies a student ID.

Curriculum separates Program, Course, Term and CourseOffering. Prerequisite edges reference courses twice; recursive reachability rejects cycles. Registration checks enrollment ownership and program/term compatibility. A prerequisite passes if a published assessment percentage average in any offering is at least 50. Offering capacity is stored but not used to reject registration. Course/program update/delete UI and APIs are not supplied by the present create/list surface.

4.2 Teaching, reporting and integrity

Assigned instructors and admin/super-admin may manage teaching records. Attendance stores sessions and student presence, but mark entry does not check registration membership. Grades use decimal persistence, explicit publication and student-only published visibility. Appeal decisions allow submitted → under-review/accepted/rejected and under-review → accepted/rejected. A changed accepted score writes GradeAuditLog with old/new value and actor. Ordinary score edits are still possible without that appeal audit path.

Exam creation/rescheduling obtains an institution/term advisory transaction lock and tests interval overlap for room, instructor and shared registered students. Back-to-back intervals are permitted. Conflict search is term-scoped, not institution-wide across all terms. PDF transcripts average published percentages; attendance summaries use all created sessions. At-risk status is attendance below 75 percent OR the average of the latest two published assessment percentages below 50; fewer than two grades does not produce a grade signal.

4.3 Broker transfer

The academic worker polls every two seconds, up to twenty unpublished rows, with `FORUPDATESKIPLOCKED`. Persistent publish uses RabbitMQ confirms before setting `published_at`. Confirm-before-commit crashes may repeat delivery. Finance prefetches ten events, inserts an inbox event ID and generates an invoice from its fee schedule in one local transaction. Duplicate event/enrollment paths acknowledge without another invoice. Missing fees reject to the DLQ; transient failures requeue. There is no distributed transaction or exactly-once broker guarantee. ACAD-01–11 and FIN-02 map to Figures 8, 12, 14 and 21.

5 Finance, payments and analytical projections

5.1 Payment intent and reconciliation

Finance owns the invoice amount in whole XAF. A `PaymentIntent` stores invoice ID, provider reference, amount, mobile number and pending/succeeded/failed status. A partial unique index permits at most one pending/-succeeded intent per invoice. The API commits the intent before calling the provider, preserving evidence if external I/O fails. It does not claim success on provider request acceptance.

`PaymentGateway` is a Python Protocol implemented by `MtnMomoGateway` and `CamerPayTestDoubleGateway`. The fake is stateless: numbers ending in 0000 fail; otherwise a six-second elapsed interval produces success. It works across API/worker processes without shared memory. The MTN adapter uses a token endpoint and collection request/status endpoints with a fifteen-second HTTP timeout; live sandbox validation is not established. The poller queries status and does not resubmit a request never received by MTN.

5.2 ACID financial effect

Reconciliation locks the intent, short-circuits terminal state and asks the provider. Failure commits failed intent only. Success locks the invoice and, if not already paid, changes status, posts equal cash debit and tuition revenue credit under one `posting_id`, and inserts a receipt unique by invoice ID. These changes commit together. There is no separate ledger-transaction table. Balance is enforced by the application helper, not a cross-row database CHECK constraint. Receipt references are UUID columns with invoice uniqueness, not foreign keys.

The worker runs payment reconciliation every three seconds and hourly summary catch-up alongside RabbitMQ consumption. Student UI polls active payment state at 1.5 seconds; success invalidates the invoice query. A provider callback is only a status-query hint and currently cannot traverse the public gateway without bearer auth. Provider I/O inside the reconciliation row-lock lifetime affects contention.

5.3 Expenses, campaigns and summaries

Expense creation posts expense debit and cash credit atomically. Campaign ROI is attributed paid invoice revenue minus campaign cost, divided by cost; zero cost returns null with a reason. Attribution is a simple converted-lead/student join, not an exclusive marketing attribution model; duplicate lead links can inflate totals. Monthly summaries aggregate ledger timestamps in half-open month intervals and append numbered versions. Hourly catch-up covers fully elapsed missing months since first ledger activity. Neither campaign revenue nor pending invoice totals replace posted ledger revenue. FIN-01–12 map to Figures 6, 9 and 13.

6 HR workflows and payroll computation

6.1 People and self-service

Recruitment stores positions and candidate stages. Hiring creates Employee and marks Candidate hired in one commit, leaving identity linkage null. General stage update accepts enum values without a recruitment transition graph; the hire path rejects an already-hired candidate. There is no complete employee CRUD or identity provisioning API. Staff self-service resolves employee from the verified user's UUID.

AttendanceShift belongs to Employee and stores the current QR token ID. Issuing a token replaces that ID and gives a ten-minute token using a distinct HS256 secret/audience. Check-in validates expiry/signature/current ID, employee ownership and shift/employee uniqueness. The UI accepts pasted token text; camera/biometric integration and actual presence verification are absent.

Leave approval rejects self-approval and final-state redecision; decision and notification commit together for linked users. Notifications are fetched records, not real-time push. Performance reviews record actor, period, rating and comments. Asset movement locks Asset and checks aggregate assignment against stock; employee-specific return balance and employee-reference validation are not additional guarantees. Administrative screens remain placeholders.

6.2 Exact payroll algorithm

Let G be gross monthly XAF, C the CNPS ceiling, r_e, r_m employee/employer rates, and d the standard deduction. The implementation computes:

$$B = \min(G, C), \quad E = Br_e, \quad M = Br_m, \quad T = \max(0, (G - E)(1 - d)).$$

For ascending marginal brackets with lower/upper bounds L_j, U_j and rate r_j :

$$I = \sum_j \max(0, \min(T, U_j) - L_j) r_j, \quad N = G - \text{round}_{HU}(E) - \text{round}_{HU}(I).$$

The final open bracket has no fixed upper limit. CNPS, taxable base and tax are reported using whole-franc HALF_UP rounding; tax is computed on the unrounded intermediate taxable value. Employer CNPS is reported separately, not deducted from employee net.

6.3 Release gate

A PayrollRun snapshots active-employee payslips for a unique institution/period. Only a draft backed by a manually verified schedule can be approved; only approved runs appear in employee self-service. Rate verification is a super-admin attestation, not validation against official CNPS/DGI publications. The selected version is explicit; the code does not automatically select the legally applicable version by run date. No payroll funds or finance event are sent. HR-01–11 map to Figures 10, 15, 16 and 22.

7 Deployment, configuration and delivery pipeline

7.1 Local runtime

Compose creates ten containers: four web APIs, two workers, frontend, gateway, PostgreSQL and RabbitMQ. API images use Python 3.13; frontend is built with Node 22 and served by Nginx 1.27. Service APIs listen on 8000 internally, frontend/gateway on 80, database on 5432 and broker AMQP on 5672. Local gateway publishes 8081; broker management publishes loopback 15672. Domain APIs/databases are not public host listeners in the base stack. The test overlay changes development test exposure.

The frontend Dockerfile runs `npm ci` and the TypeScript/Vite build, then copies `dist` into Nginx. Static Nginx provides SPA deep-link fallback and thirty-day immutable caching for selected asset extensions. Vite development proxies `/api` to the gateway on 8081; gateway-root `/healthz` is not covered by that Vite proxy. Keys are read-only mounted; production secrets must not use dev key material.

7.2 Delivery definitions versus execution evidence

Backend Jenkins defines contract tests, shared test PostgreSQL/key setup, a four-service lint/type/test/build matrix and database isolation tests. Coverage is gated at 70 percent. Frontend Jenkins defines install, lint/typecheck, coverage, application build and image build; its coverage configuration does not explicitly enforce the same numeric threshold. Neither root CI pipeline pushes images. The separate backend-owned promotion pipeline expects already published images and a paired frontend ref.

Promotion specifies candidate contracts/E2E, human approval, image pull, backup gate, migrations, deploy, smoke and release recording. Controller validation, credentials, webhooks, registry publication, exact image provenance and live execution remain unverified. Calling a tag “immutable” does not enforce registry immutability, and the image template uses colon tag syntax rather than a dedicated digest field.

7.3 Production configuration gates

The override intends loopback gateway binding, pinned registry images and external keys. Environment and port merge semantics must be validated, not inferred from comments: development mode and insecure-cookie setting remain inherited. Host Nginx/TLS and domain provisioning are external prerequisites. Readiness endpoints check database connectivity; current deploy and monitoring loops primarily use liveness and do not establish worker or broker readiness. OPS-01–03 and UI-05 map to Figure 17 and Appendix G.

8 Operational qualities, failure handling and evolution

8.1 Observability and capacity

Application logging uses Python logging, plus Nginx access/error logs with request duration. This is not a deployed central log-aggregation service. A scheduled script checks gateway/service liveness, any enrollment DLQ messages, backlog above 100, disk at 85 percent and memory at 90 percent by default. It has no worker heartbeat, WAL age or backup freshness monitoring. HTTP readiness cannot certify successful message transfer.

The historical local load report records 25 VUs for 90 seconds: read p95 882.9 ms and 0.48 percent errors. The proposed target is five minutes, p95 below 500 ms and error rate below one percent. The latency target is not met. Shared-host contention and per-request connection creation with SQLAlchemy NullPool are hypotheses, not proven causes. Query plans and dedicated-host measurements must precede tuning.

8.2 Recovery and rollback

The existing isolated PITR drill uses base backup and archived WAL with an explicit target time, reporting 21.8 seconds to restore a tiny test table. It is not the full ERP cluster and does not prove production RPO/RTO. Scheduled pgBackRest, encrypted off-host repository, retention and full event/session reconciliation remain targets. Deployment backup checks inspect filesystem freshness; the default WAL-age gate of fifteen minutes is weaker than the five-minute proposed RPO objective.

Rollback is a manual image selection with schema-compatibility review; it must not be mistaken for database restoration. Production outages require an approved recovery point, isolated restore, broker/outbox/inbox reconciliation and session review before traffic returns. Figure 18 is explicitly a target runbook, not a performed exercise.

8.3 Evolution and verification

Stateless APIs and shared key/session persistence permit a conceptual replica strategy, but connection budgets, lock duration, summary-generation races, refresh concurrency and static upstream registration need validation first. Single-host replicas do not remove the host, database or broker failure domains. Figure 19 marks this as conceptual.

Prioritize missing required instructor/admin dashboards, CRUD coverage, effective production settings, credentialed-or-disclosed payment integration and evidence collection. Preserve event compatibility and explicit ownership during change. The SRS traceability matrix covers unit/integration/E2E assets; appended design decisions explain known trade-offs rather than concealing them.

A Design decisions, consistency and data rationale

A.1 Decision register

ID	Decision	Rationale, consequence and verification
DD-01	Four service databases	Domain ownership meets the brief; one cluster reduces local resource cost but shares failure/capacity. Verify database connection isolation.
DD-02	REST plus enrollment events	Interactive API contracts remain simple; outbox avoids enrollment dependence on Finance availability. UI must represent eventual invoice creation.
DD-03	Transactional outbox/inbox	Local commits plus idempotent consumer prevent ordinary lost/duplicate business effects. Publish-before-mark crash replay is expected.
DD-04	RS256 and rotating cookies	Resource services need only public key; refresh-token hashes reduce raw-token exposure. Stateless access tokens cannot be immediately revoked by refresh logout alone.
DD-05	Forced RLS + app context	Database policies add tenant enforcement to API checks; owner roles require FORCE. Internal platform contexts and non-RLS plumbing are explicit exceptions.
DD-06	Protocol-based provider adapter	Real and deterministic providers share request/status contracts; fake is stateless across processes. No live-sandbox evidence is claimed.
DD-07	Whole-XAF money	Integers avoid binary decimal-money error; Decimal supports payroll rates. Browser numbers must stay within JavaScript safe integer bounds; no numeric boundary policy is yet enforced across all surfaces.
DD-08	Financial snapshots	Receipts, payslips and numbered monthly summaries preserve historical output at creation. These intentional derived records are not a claim of strict normalization of every relation.
DD-09	Local advisory/row locks	Exam term lock, payment row locks and asset lock protect specific invariants. Other workflows cannot inherit concurrency guarantees from these examples.
DD-10	React Query + service adapters	Separates server cache, form state and transport shape. Per-principal cache invalidation and full runtime response validation remain gaps.
DD-11	Manual payroll attestation	Prevents unverified schedules from backing approval; requires human statutory evidence. Boolean attestation does not preserve who verified or authoritative source provenance.
DD-12	Single-host Compose	Practical local and constrained-host demonstration; Kubernetes and full monitoring cluster are not prerequisites. Production isolation/TLS/backup still require configuration.
DD-13	Explicit partial delivery	Placeholder pages communicate absence rather than fake analytics. Required dashboards and lifecycle APIs remain acceptance work, not silently excluded scope.

A.2 Normalization and denormalization

Institution, campus and user facts are separated in Identity. Academic separates program, course and term from offering, enrollment, registration and assessment. Prerequisite edges resolve the course-to-course relationship without repeated lists. Finance separates fee configuration from issued invoices and payment attempts. HR separates recruitment, employee, shift, leave, review, schedule and run. These structures reduce update anomalies and generally follow dependency separation appropriate to 3NF-oriented transactional design.

The model intentionally retains repeated `institution_id` for tenant policy evaluation. Refresh sessions copy tenant identity from user at issue time. Invoice amount snapshots a configured fee; payroll payslips snapshot computed amounts; financial summaries store versioned aggregates. Outbox payloads and payroll brackets use JSONB/list-shaped data. Such structures need invariants and provenance rather than a blanket assertion that every relation is strictly in 3NF. No formal functional-dependency proof or EXPLAIN comparison is supplied in the baseline.

A.3 Key constraints and ownership exceptions

- **Identity:** institution slug, user email and refresh-token hash uniqueness; nullable user institution/campus for platform use.
- **Academic:** enrollment student/program/term; course program/code within institution; prerequisite edge; offering course/term; grade assessment/student; exam one-per-offering.
- **Finance:** fee tenant/program/term; invoice tenant/enrollment; inbox event primary key; provider reference; partial active-intent uniqueness; receipt invoice uniqueness; summary tenant/period/version.
- **HR:** payroll tenant/period; payslip tenant/employee/period; attendance shift/employee. Some same-database identifiers intentionally lack foreign keys, including payslip employee, asset-movement employee and receipt references.

Cross-service user/student/campus/program/enrollment IDs remain opaque UUIDs. An ERD dotted relationship is a logical lookup, not a database-enforced FK. The generated dictionary records only declared column metadata; migrations remain authoritative for composite/partial indexes and RLS policies.

A.4 Transaction and concurrency matrix

Workflow	Atomic unit / lock	Residual qualification
Enrollment	Enrollment + outbox commit	Duplicate pre-check alone is not the concurrency boundary; DB uniqueness is final.
Outbox publish	Row locks, confirms, publication timestamp	Confirm and DB commit are separate durability systems; replay possible.
Invoice consumption	Inbox + invoice commit	Missing fees roll back and dead-letter; unsupported-version enforcement incomplete.
Payment settlement	Intent/invoice row locks; status + ledger + receipt	External status I/O extends lock lifetime.
Expense	Expense + balanced ledger commit	Balance is application-checked, not a cross-row database constraint.
Exam	Institution/term advisory transaction lock	No cross-term resource check.
Grade appeal	Decision + changed score + audit commit	No explicit row lock in decision path; direct grade edits bypass appeal audit.
Leave	Decision + notification commit	Concurrent approval race needs independent acceptance testing.
Payroll run	Run + all payslips commit	Statutory validation/manual approval are separate operations.
Assets	Asset row lock + movement commit	Aggregate stock protected; per-employee over-return is not independently checked.

B Detailed frontend integration and UX coverage

B.1 Frontend-to-API mapping

Component / service	HTTP interaction	Refresh behavior and limitation
AuthProvider	POST <code>/auth/refresh</code> ; GET <code>/auth/me</code>	Restore on mount; principal set after both succeed.
LoginPage	POST <code>/auth/login</code> ; GET <code>/auth/me</code>	Form validation precedes request; redirects to remembered pathname.
AppShell	POST <code>/auth/logout</code>	Access memory cleared by service finally; shared query cache is not cleared.
EnrollmentPage	GET academic programs/terms/enrollments; POST enrollments	Invalidates enrollments; new enrollment invoice polls at 2 s. Non-student queries can start before the placeholder branch.
InvoiceCell	GET finance invoices filtered by enrollment ID	Null means invoice not yet generated; only just-created enrollment has interval polling.
CoursesPanel	GET courses, offerings, registrations, own at-risk; POST registration	Invalidate registrations and at-risk on successful registration.
CourseGrades	GET offering assessments, then assessment grades; POST grade appeal	Lazy queries when expanded; submitted-appeal indication is local component state, not loaded appeal history.
Academic reports	GET transcript / attendance-summary	Blob download with fixed file names; rejected downloads lack a complete UI error flow.
FinancePage	GET finance invoices	Student-only displayed content; invalidates own invoices on payment success.
PayForm	POST invoice payment-intent; GET payment-intent	Local intent ID; 1.5 s poll when data status is pending; failure allows retry. No reload restoration of pending intent selection.
Receipt	GET invoice receipt	Offered for paid row; PDF download.
LeaveSection	GET leave requests/mine; POST leave requests	Invalidates own leave list; no automatic live approval updates.
AttendanceSection	GET attendance/mine; POST attendance/check-in	Manual token entry; invalidate attendance after success.
PayslipsSection	GET payroll/payslips/mine	Displays saved whole-XAF gross/CNPS/IRPP/net; no approval UI.
NotificationsSection	GET notifications/mine; POST notification/read	Invalidates notification query on read; no timed polling or push.
SystemStatusPage	GET gateway and service <code>healthz</code> paths	Five liveness cards; page reload rechecks. No readiness or worker state.

All business paths above begin with `/api/v1`; prefixes are abbreviated for readability. Identity uses `/auth`, HR `/hr`, Finance `/finance` and Academic `/academic`. See service adapter files for exact path assembly.

B.2 UI role matrix

Page	Student	Staff	Admin / super-admin
Overview / Status	Visible	Visible	Visible; overview is welcome-only.
Academic	Functional self-service	Placeholder	Placeholder.
Finance	Functional payment portal	Placeholder	Placeholder.
People	Placeholder	Functional own workspace	Placeholder despite backend admin capabilities.
Settings	Placeholder	Placeholder	Placeholder.

All navigation items are rendered to authenticated users. This is explicit incomplete feature delivery, not a permission model implemented through hiding links. The frontend's four role values match backend roles, but

department-specific role separation is not implemented.

B.3 Response/error contracts and data lifecycle

HTTP adapters translate fields, not domain rules. Pydantic is server validation; Zod covers selected input forms. The generic JSON helper uses type assertions and does not validate arbitrary server responses at runtime. A structured `detail` object can be reduced to an unhelpful string by generic error formatting. Payment-poll request errors may leave a waiting message; individual grade-query errors can resemble “Not graded”. Mark-read mutation errors and download promise failures need dedicated user feedback.

Query cache uses one application-level client, while access identity changes in AuthProvider. A robust next revision should cancel/clear principal-owned queries on logout/session replacement and include stable principal scope where appropriate. That is a target design, not an implemented property of this baseline.

B.4 Accessibility and responsive verification scope

Source supplies visible form labels, focus styles, a skip link, loading status indicators and reduced-motion support on login/status animations. Manual attendance code entry avoids dependence on a camera. Existing E2E assets check login, keyboard flow, invalid credentials and reduced motion, plus a shared authenticated staff session across overview, people, finance placeholder and status at four viewports: 360x800, 768x1024, 1440x900 and 1920x1080. The suite deliberately avoids reusing one rotating cookie in many browser contexts. It is not coverage of every student/admin workflow or full WCAG conformance. Desktop logout placement, mobile focus/menu behavior and missing error paths need manual review. No fresh E2E run is claimed by this documentation build.

C Message, API and numerical contracts

C.1 EnrollmentAccepted.v1 envelope

```
{
  "event_id": "<outbox UUID>",
  "event_type": "academic.enrollment_accepted",
  "schema_version": 1,
  "tenant_id": "<institution UUID>",
  "occurred_at": "<UTC ISO-8601 timestamp>",
  "correlation_id": "<workflow UUID>",
  "data": {
    "enrollment_id": "<UUID>",
    "student_id": "<UUID>",
    "program_id": "<UUID>",
    "term_id": "<UUID>",
    "campus_id": "<UUID>"
  }
}
```

The placeholders are explanatory, not a schema-valid fixture. Use the checked-in `contracts/events/examples/enrollment-accepted.v1.example.json` for executable contract validation. RabbitMQ routing key carries `.v1`; envelope event type does not. Exchange `erp.events` is durable topic; queue `finance.enrollment_accepted` binds `academic.enrollment_accepted.*`. Rejected messages route via direct `erp.dlx` to `finance.enrollment_accepted.dlq`.

Consumer qualification: the runtime parses selected fields and UUIDs but does not fully validate schema version/type, envelope shape and every schema field. Contract-example tests are separate from runtime consumer validation. Broker authentication/topology are the trust boundary; the envelope is not cryptographically signed. Ordering across workers is not a global business guarantee.

C.2 API examples and status semantics

```
POST /api/v1/academic/enrollments
Authorization: Bearer <access-token>
Content-Type: application/json
{"program_id":"<UUID>","term_id":"<UUID>","campus_id":"<UUID>"}

POST /api/v1/finance/invoices/<UUID>/payment-intents
{"payer_msisdn":"<synthetic test number>"}

POST /api/v1/hr/leave/requests
{"starts_on":"2026-10-05","ends_on":"2026-10-06","reason":"Example"}
```

Academic 201 means enrollment accepted, not invoice paid or generated. Finance intent 201 means locally persisted attempt, not provider settlement. HR approval responses record business decisions, not external notification delivery. 404 may deliberately conceal an unauthorized owned resource. The gateway can generate 429 without the application handler running. Readiness can return 503. All examples use placeholders or synthetic dates and contain no real credentials.

C.3 At-risk and finance examples

At-risk attendance uses present records divided by all sessions of an offering. Missing marks do not count present. The grade signal averages the latest two published normalized scores by assessment creation order; one grade is insufficient. For latest percentages 40 and 55, mean 47.5 flags risk even though both individual grades are not failures. Scores 40 and 60 average 50 and do not trigger that signal. Exactly 75 percent attendance is not below threshold.

For campaign cost 100,000 XAF and attributed paid tuition 150,000 XAF, ROI is 0.5, conventionally displayed as 50 percent. This arithmetic example does not correct duplicate lead attribution in the query. Monthly net is posted tuition revenue credits minus expense debits; it is not an accrual balance sheet or a tax return.

C.4 Illustrative payroll calculation, not statutory rates

For a deliberately synthetic test schedule with gross 100,000, ceiling 80,000, employee rate 0.04, employer rate 0.08, deduction 0.30 and a single 10-percent tax bracket: employee CNPS is 3,200; employer CNPS 6,400;

taxable base 67,760; IRPP 6,776; net 90,024 XAF. These are invented arithmetic-test inputs, *not published Cameroonian rates*. Real usage requires verified effective-period CNPS/DGI sources and review of monthly versus annual bracket conventions.

D UML and architecture diagram suite

This suite exceeds the brief's required use-case, class, two sequence, ERD and deployment views. All diagrams are editable PlantUML. PNG renders are embedded for portable LaTeX builds; SVG renders accompany the package for zoomable inspection. Diagram sheets use A3 or A2 page sizes so dense architecture and sequence views are not reduced to unreadable A4 thumbnails; narrative and tables remain A4. Selected-field ERDs must be read with the full generated dictionary and migrations. SRS and SDD share context/use-case sources to avoid contradictory duplicate artwork.

D.1 System context

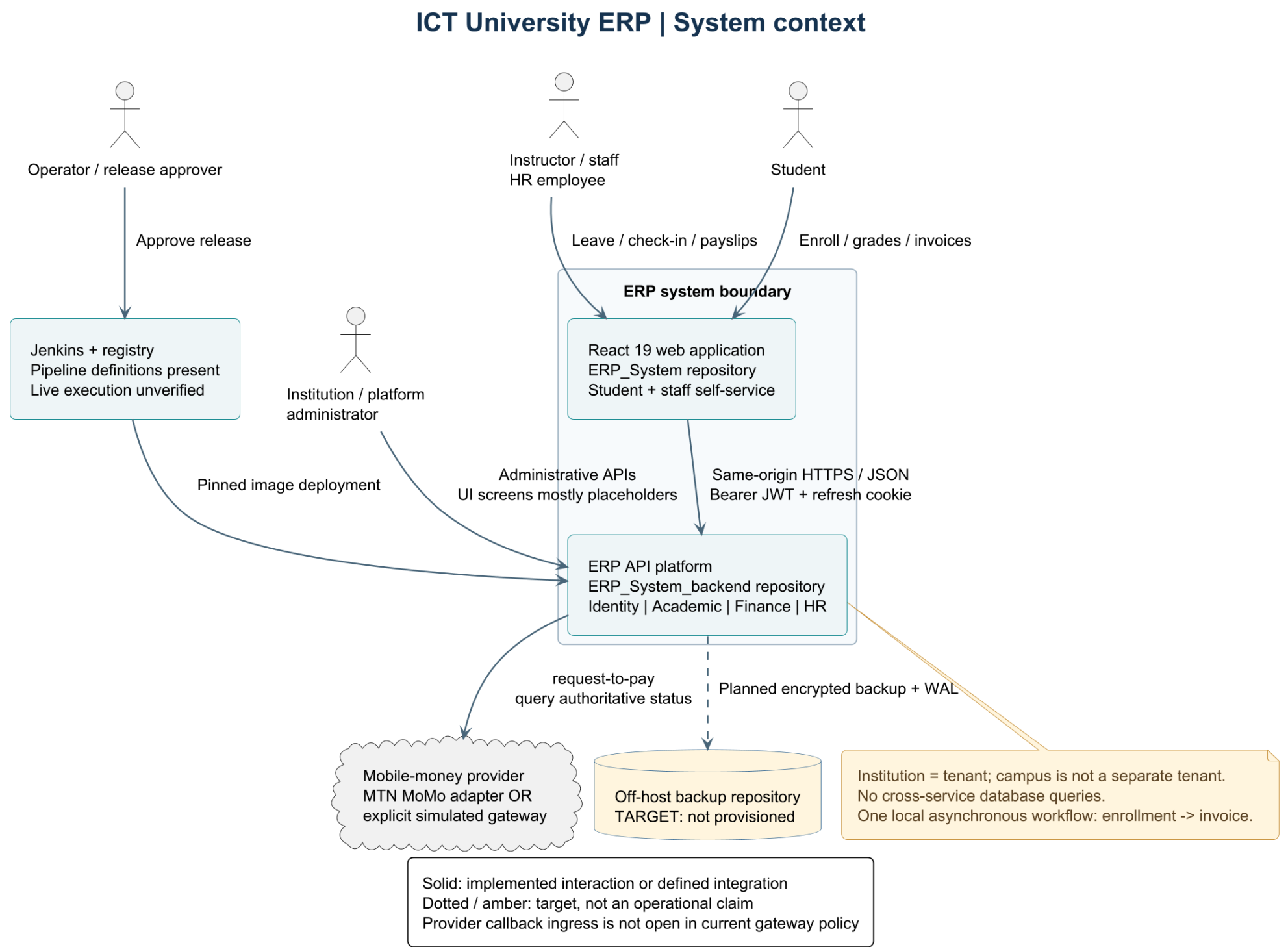


Figure 1: System context. Frontend and backend share a product boundary; external provider and operator prerequisites are explicitly identified.

Editable source: `diagrams/01-context.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.2 Actor and use-case model

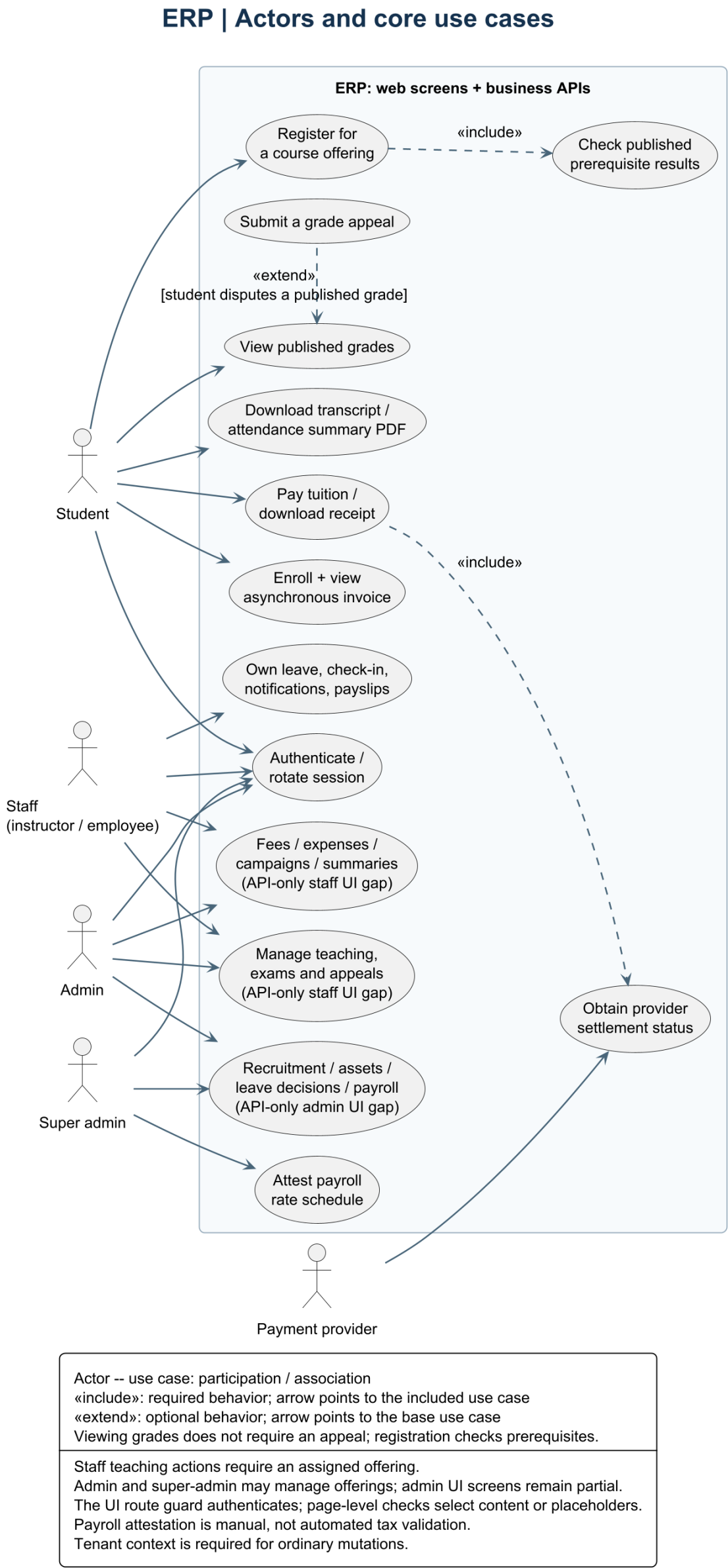


Figure 2: Actor and use-case model. Implemented APIs and self-service screens are distinguished from administrative UI gaps.

Editable source: `diagrams/02-use-cases.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.3 Runtime component architecture

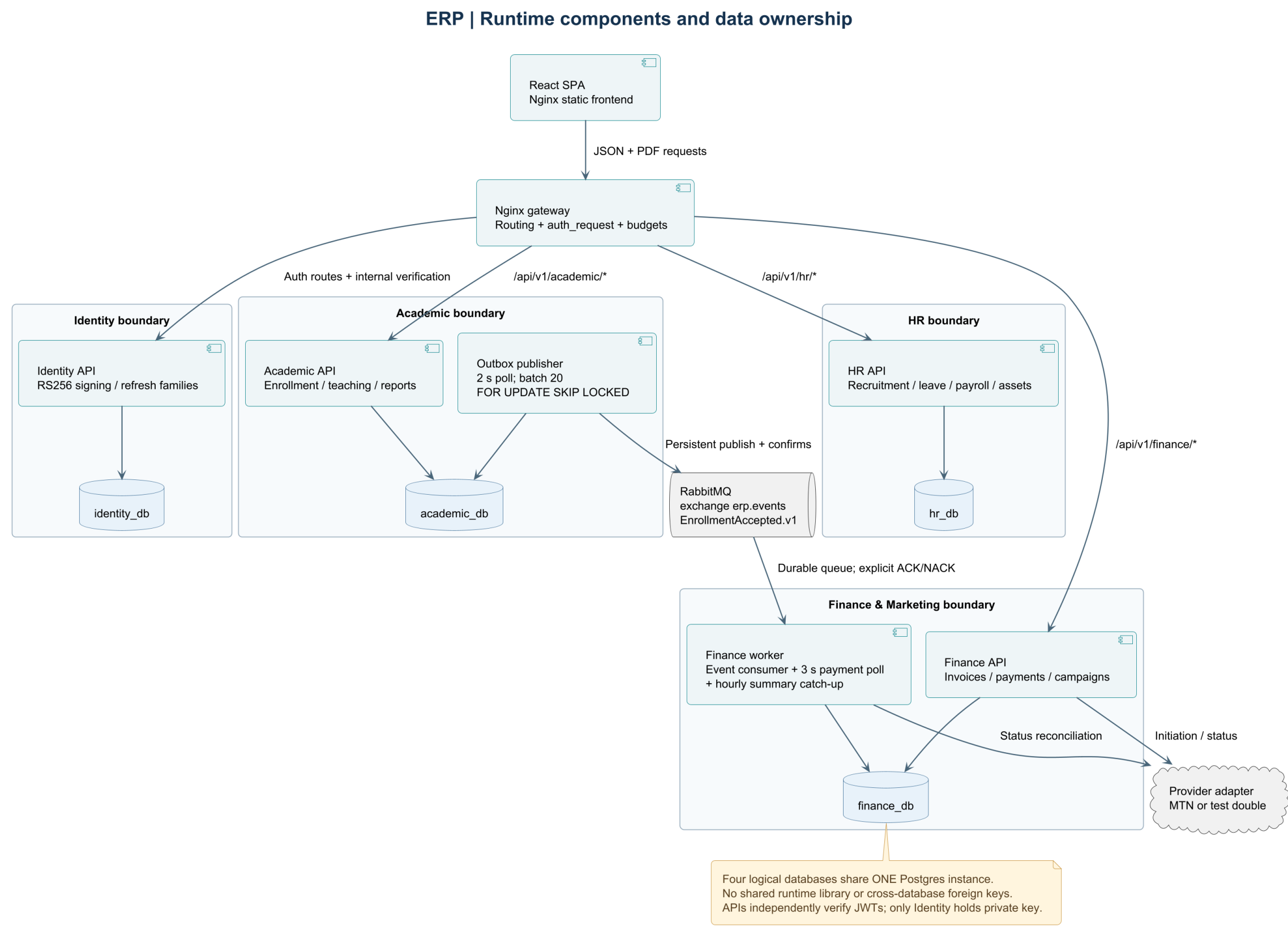


Figure 3: Runtime component architecture. Database ownership, API routing and the broker-mediated cross-service flow are explicit.

Editable source: `diagrams/03-components.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.4 Frontend composition and transport

Web application | Component and data-flow design

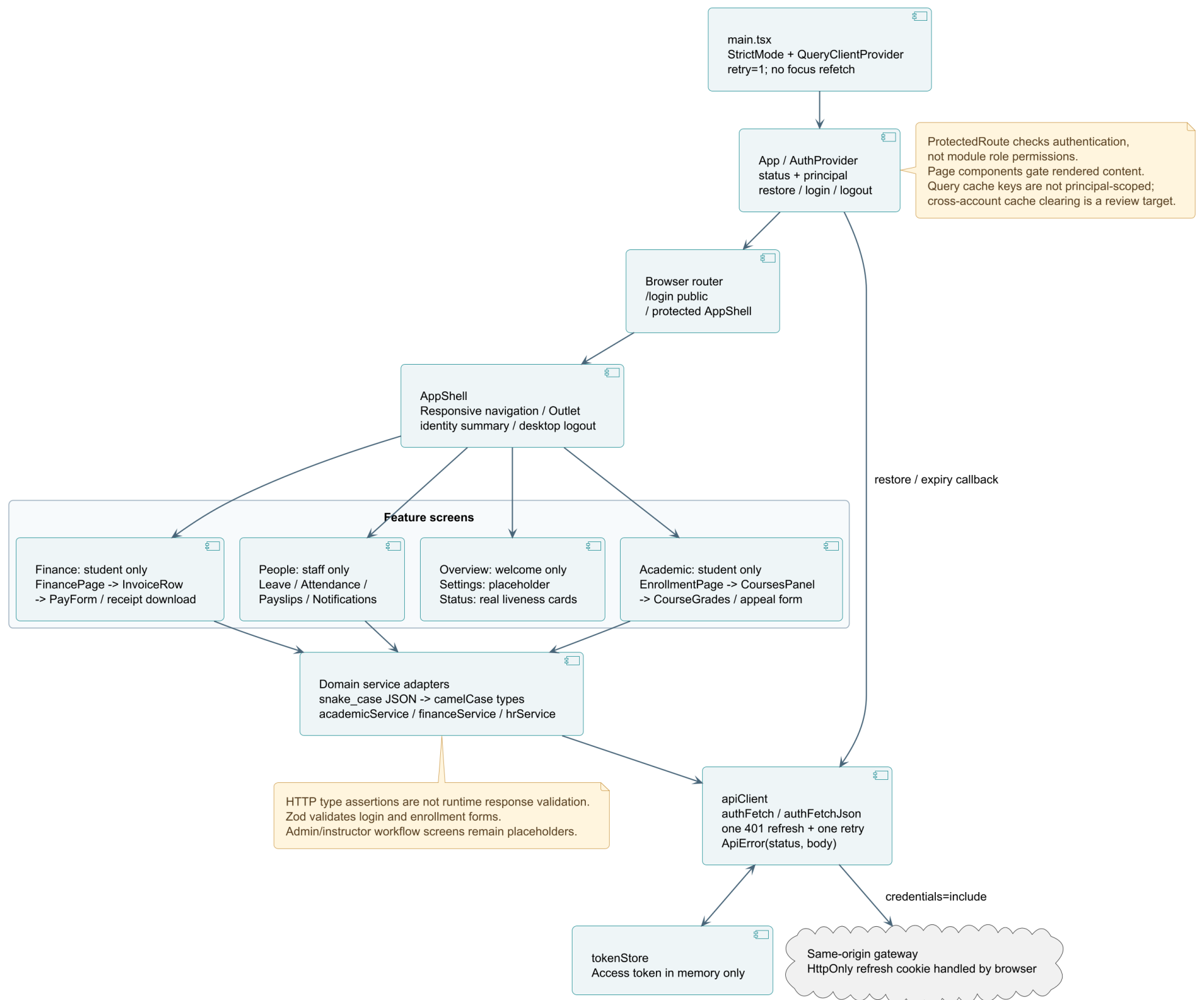


Figure 4: Frontend composition and transport. Provider, router, page, adapter, cache and token responsibilities reflect the React source.

Editable source: `diagrams/04-frontend.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.5 Full system class diagram

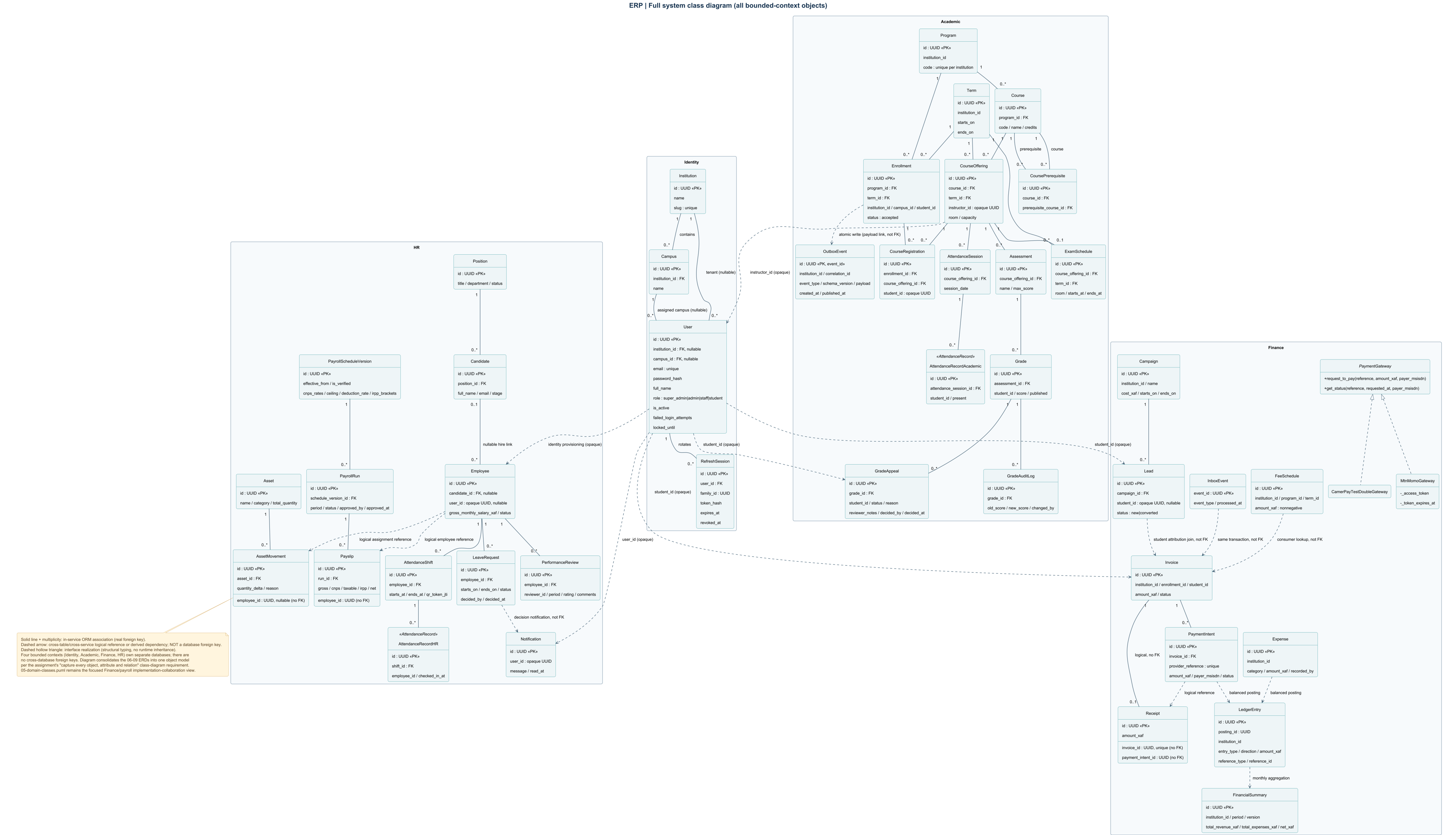


Figure 5: Full system class diagram. Every persisted class across Identity, Academic, Finance and HR with attributes, in-service associations and cross-boundary logical references consolidated into one object model.

Editable source: `diagrams/05a-full-domain-classes.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.6 Implementation class collaboration

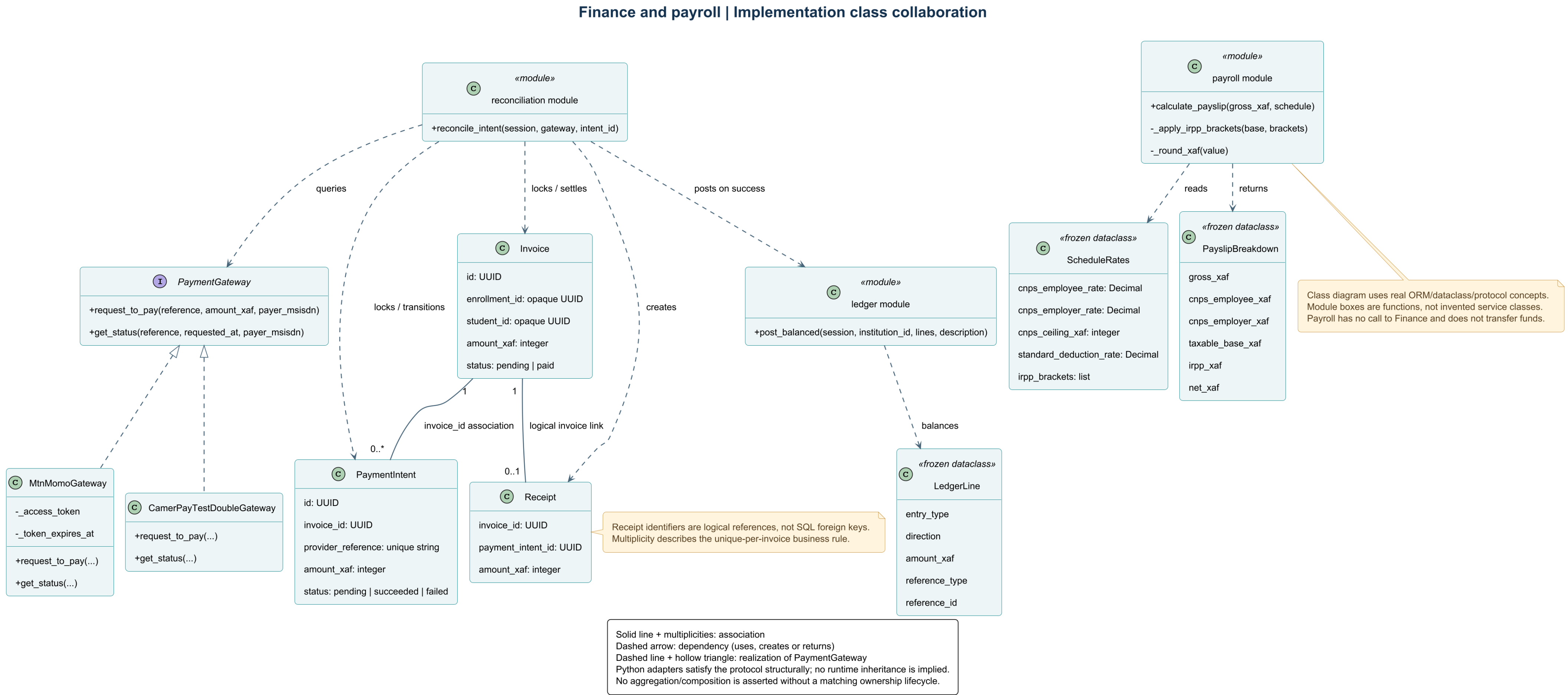


Figure 6: Implementation class collaboration. Real protocol, ORM, dataclass and module-function concepts; no invented shared service framework.

Editable source: `diagrams/05-domain-classes.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.7 Identity entity relationship model

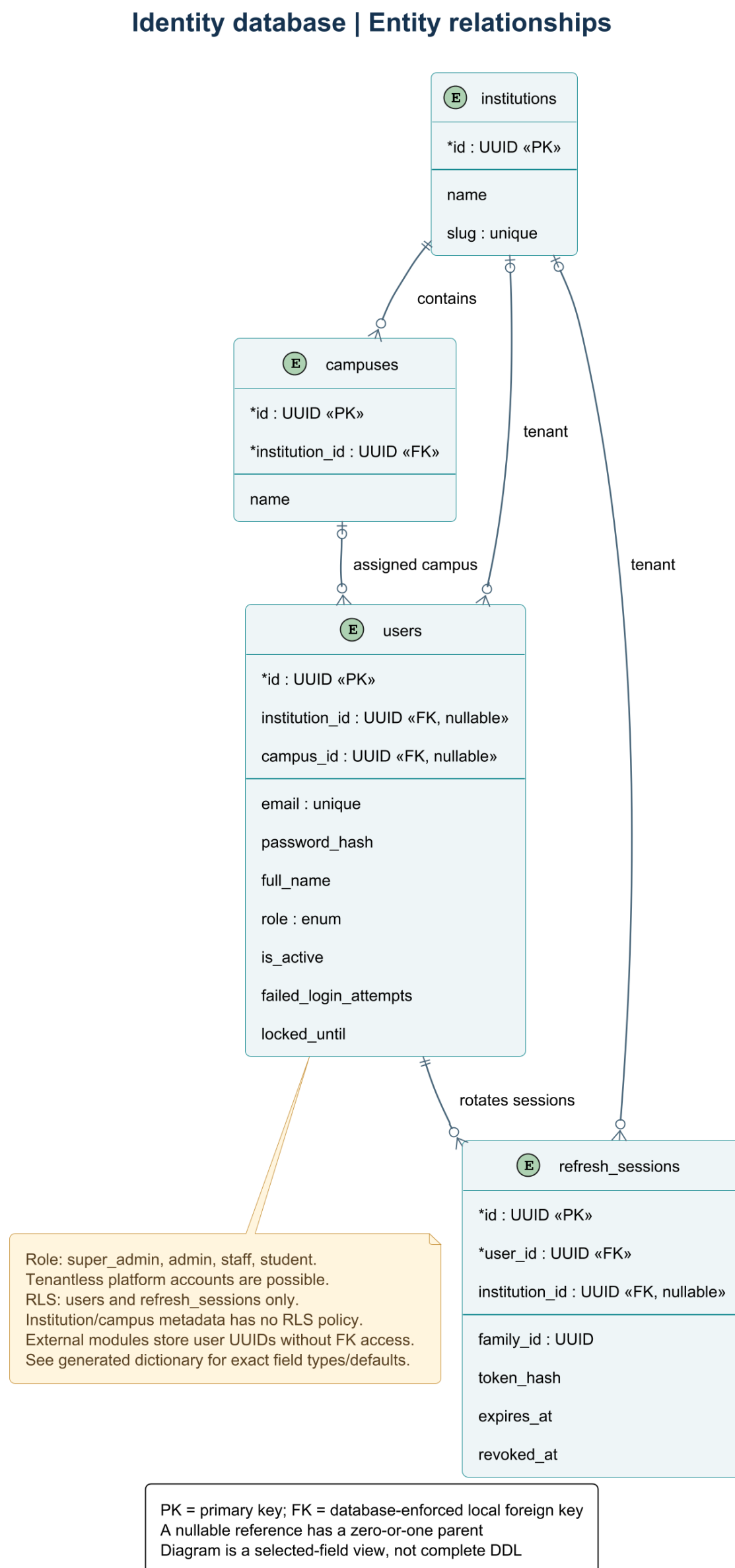


Figure 7: Identity entity relationship model. Nullable tenant/campus context and refresh-family storage are shown with local foreign keys.

Editable source: [diagrams/06-erd-identity.puml](#). This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.8 Academic entity relationship model

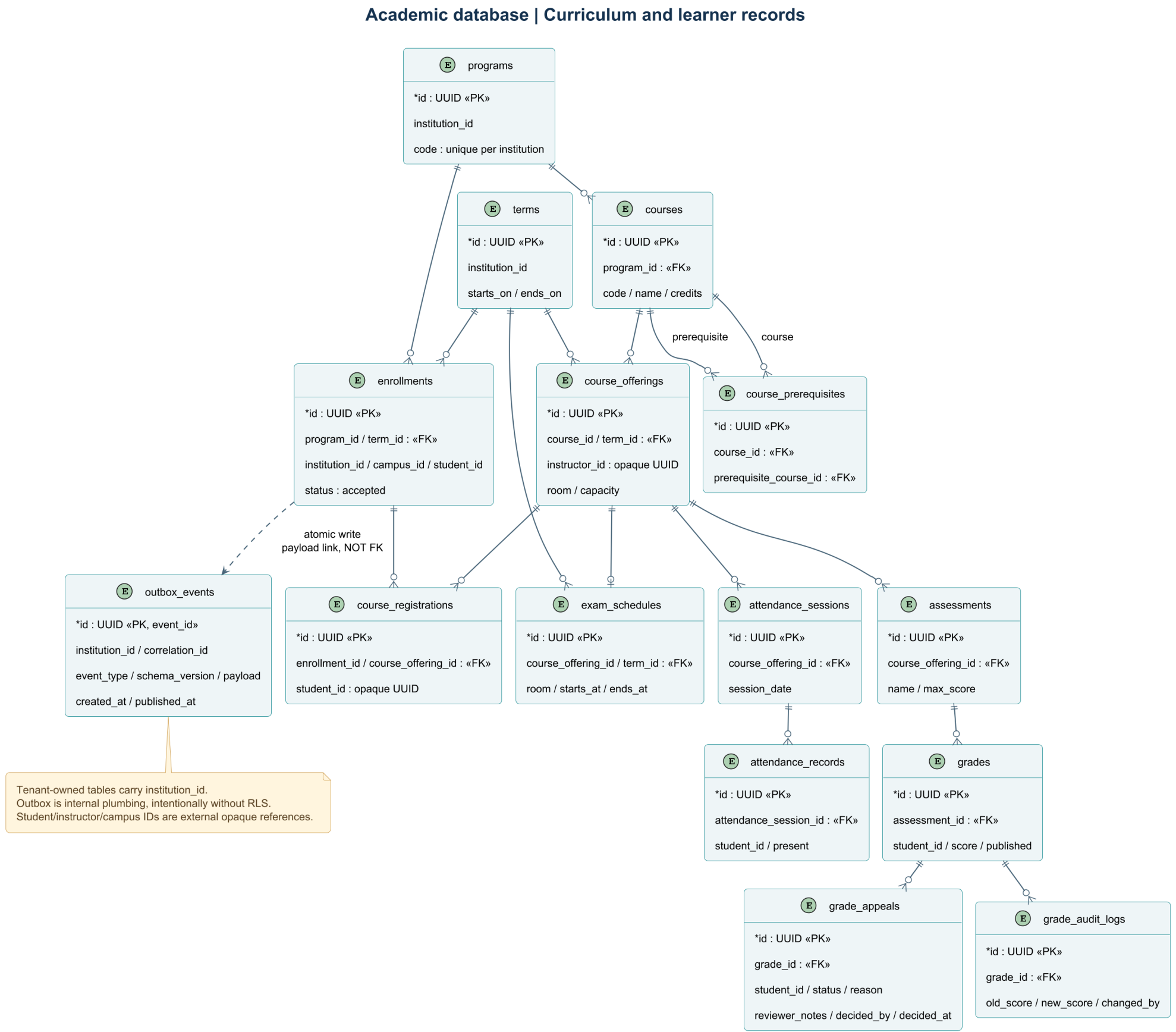


Figure 8: Academic entity relationship model. Curriculum, enrollment, teaching records and outbox are distinct; dotted outbox linkage is not a foreign key.

Editable source: `diagrams/07-erd-academic.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.9 Finance entity relationship model

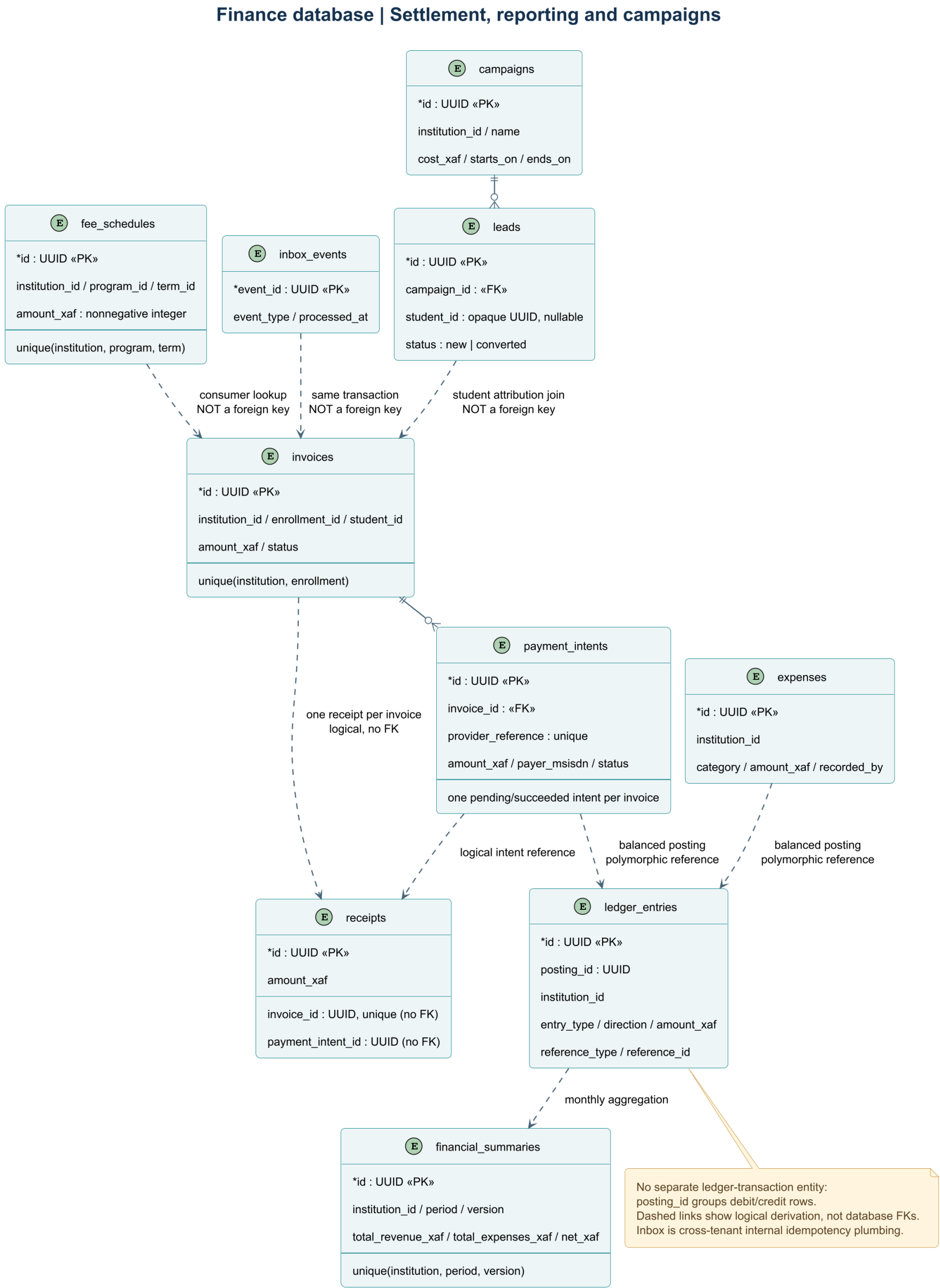


Figure 9: Finance entity relationship model. Payment and receipt associations, ledger grouping and analytical projections distinguish logical from FK relationships.

Editable source: `diagrams/08-erd-finance.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.10 HR entity relationship model

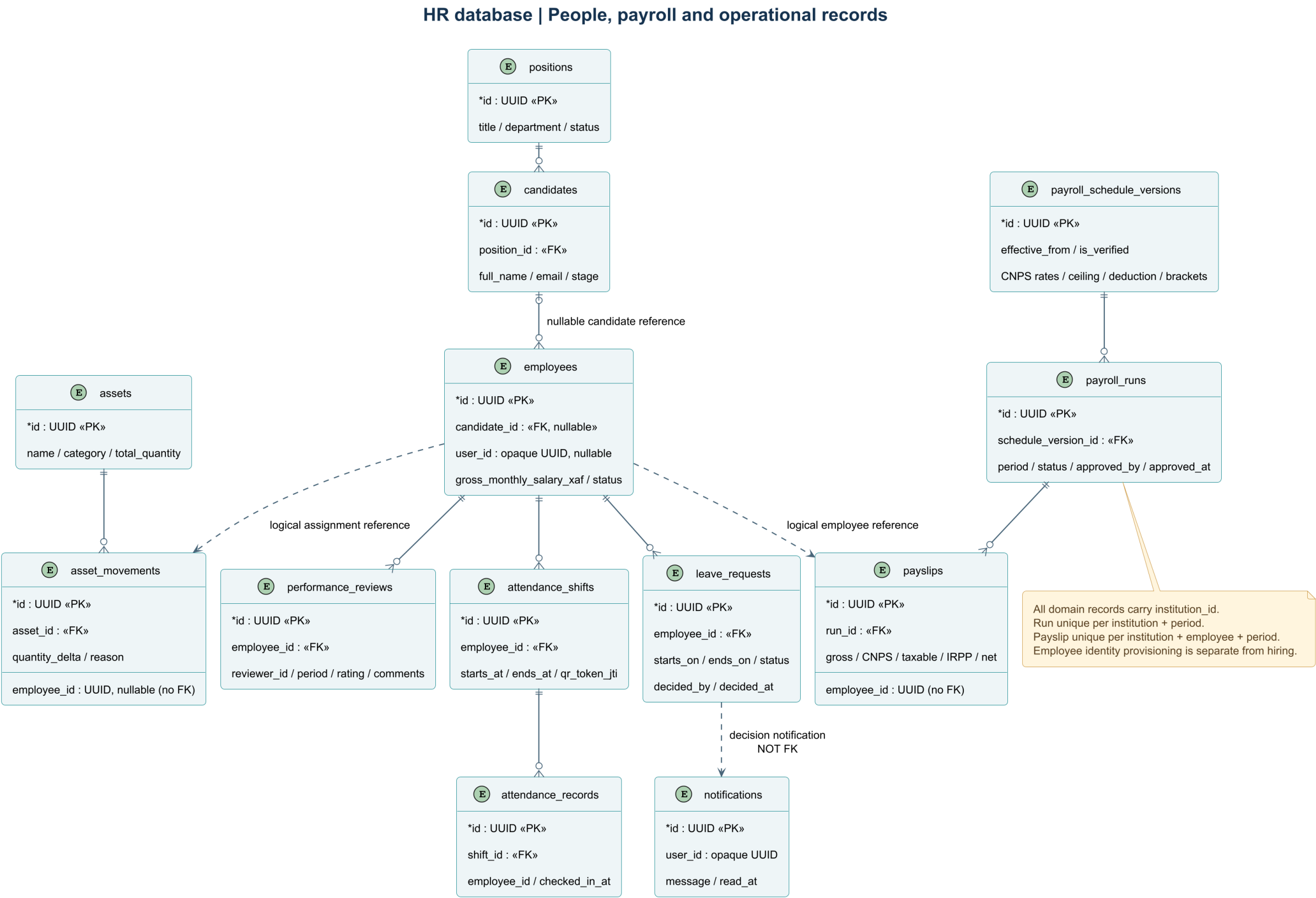


Figure 10: HR entity relationship model. Employee linkage, payroll snapshots, leave notifications and stock movements show local and unenforced references.

Editable source: `diagrams/09-erd-hr.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.11 Session authentication sequence

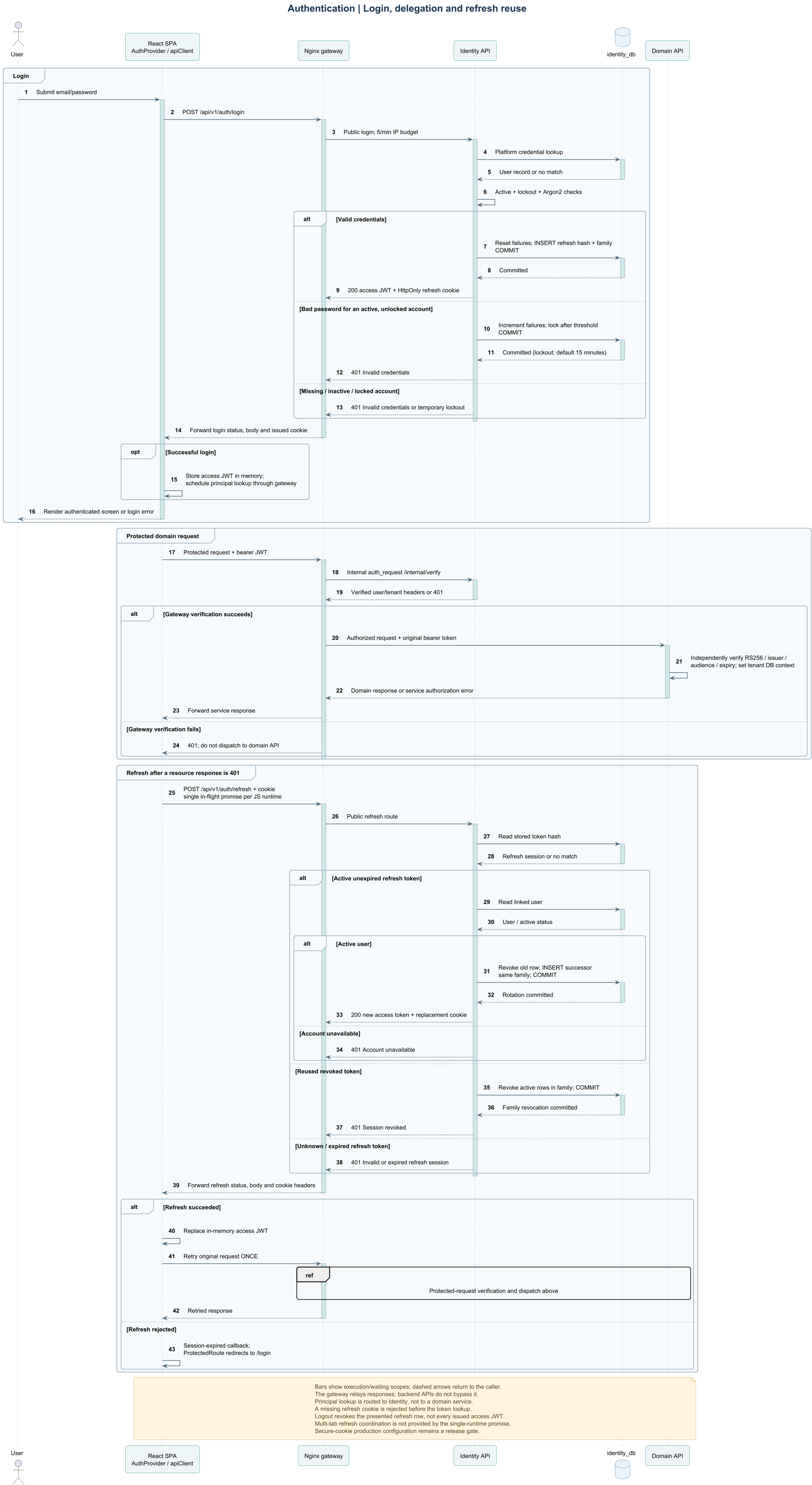


Figure 11: Session authentication sequence. Gateway delegation, independent validation, rotation and sequential reuse rejection are separate steps.

Editable source: [diagrams/10-session-sequence.puml](#). This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.12 Enrollment-to-invoice sequence

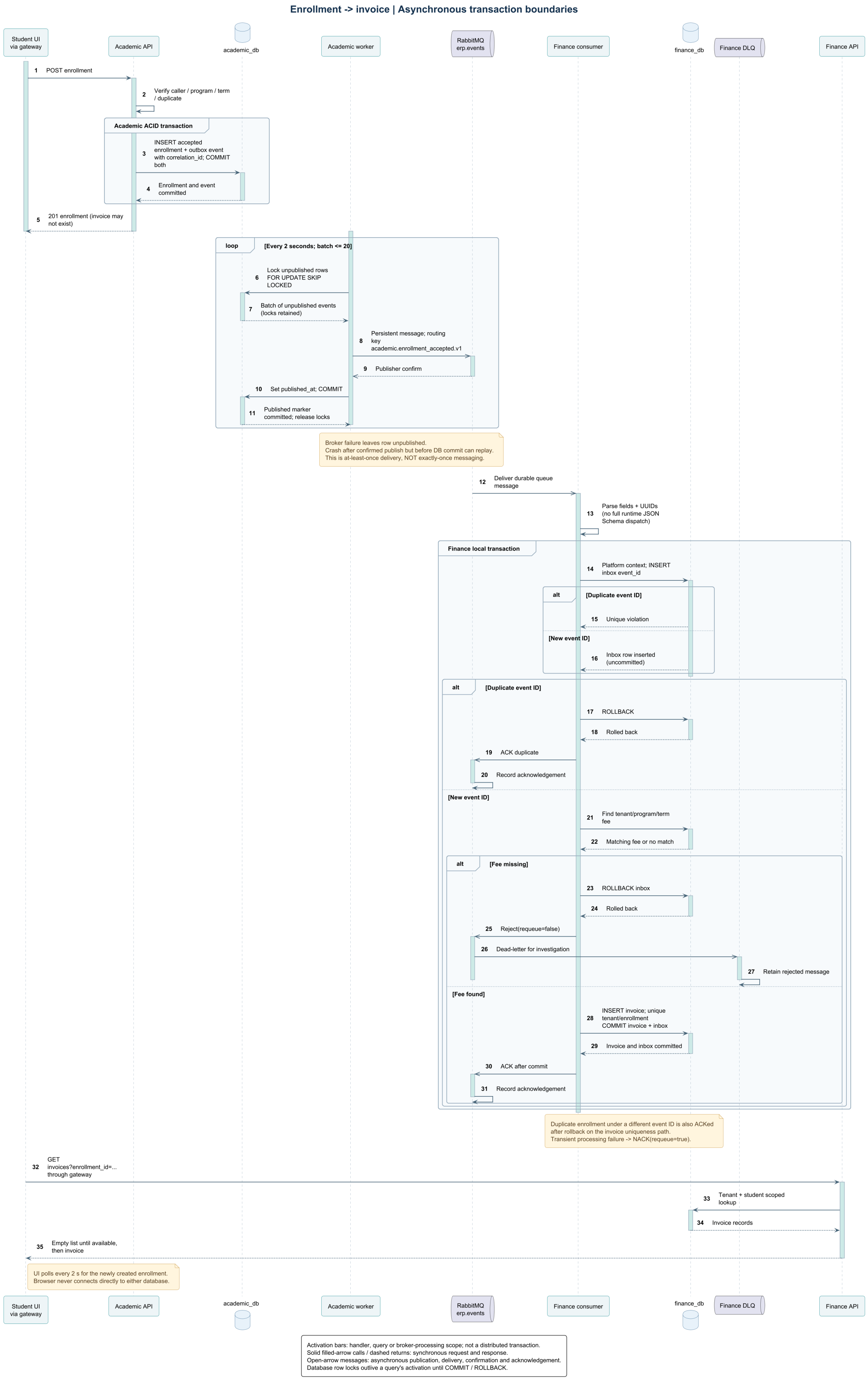


Figure 12: Enrollment-to-invoice sequence. Two local ACID transactions surround at-least-once broker transfer and explicit failure paths.

Editable source: `diagrams/11-enrollment-sequence.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.13 Payment settlement sequence

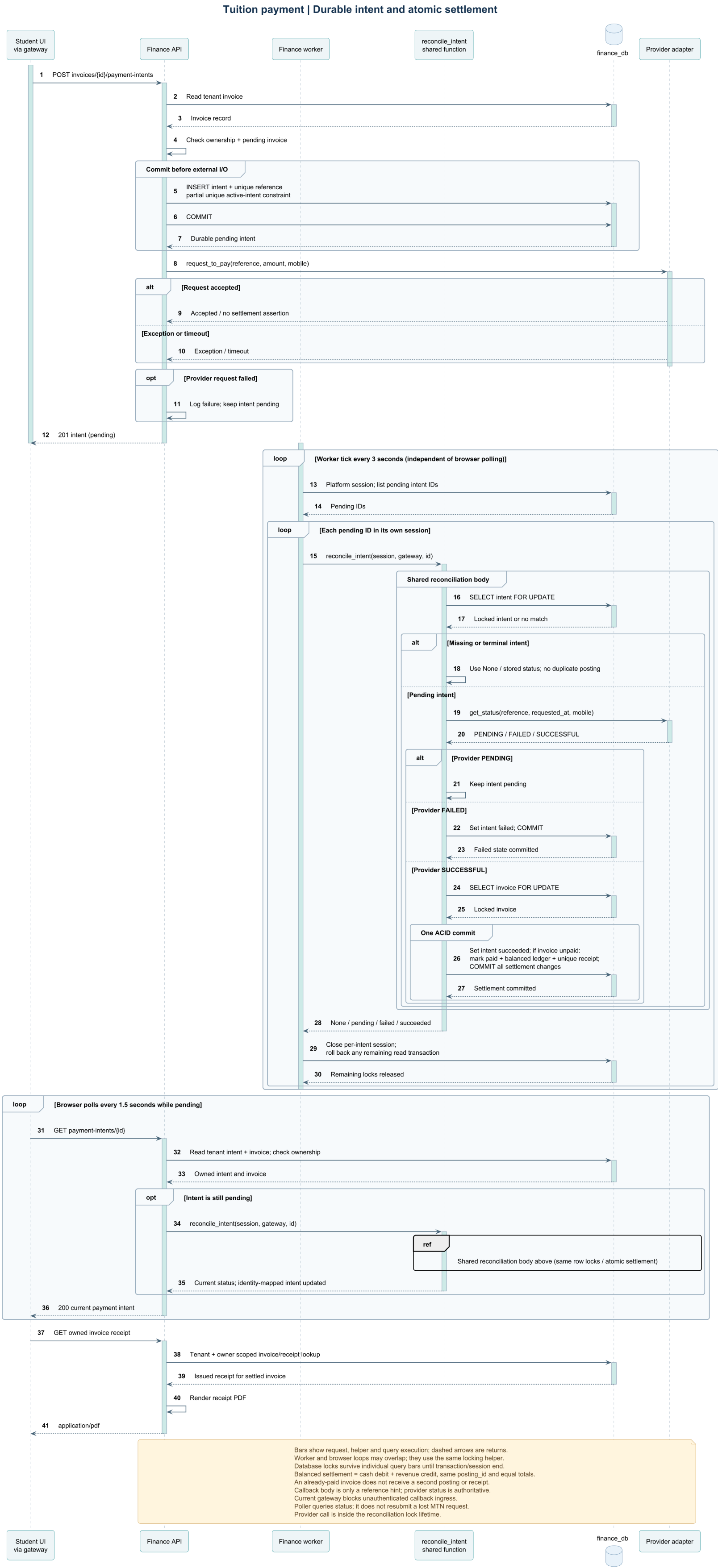


Figure 13: Payment settlement sequence. Durable intent precedes provider I/O; reconciliation atomically applies the financial effect.

Editable source: [diagrams/12-payment-sequence.puml](#). This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.14 Grade appeal state machine

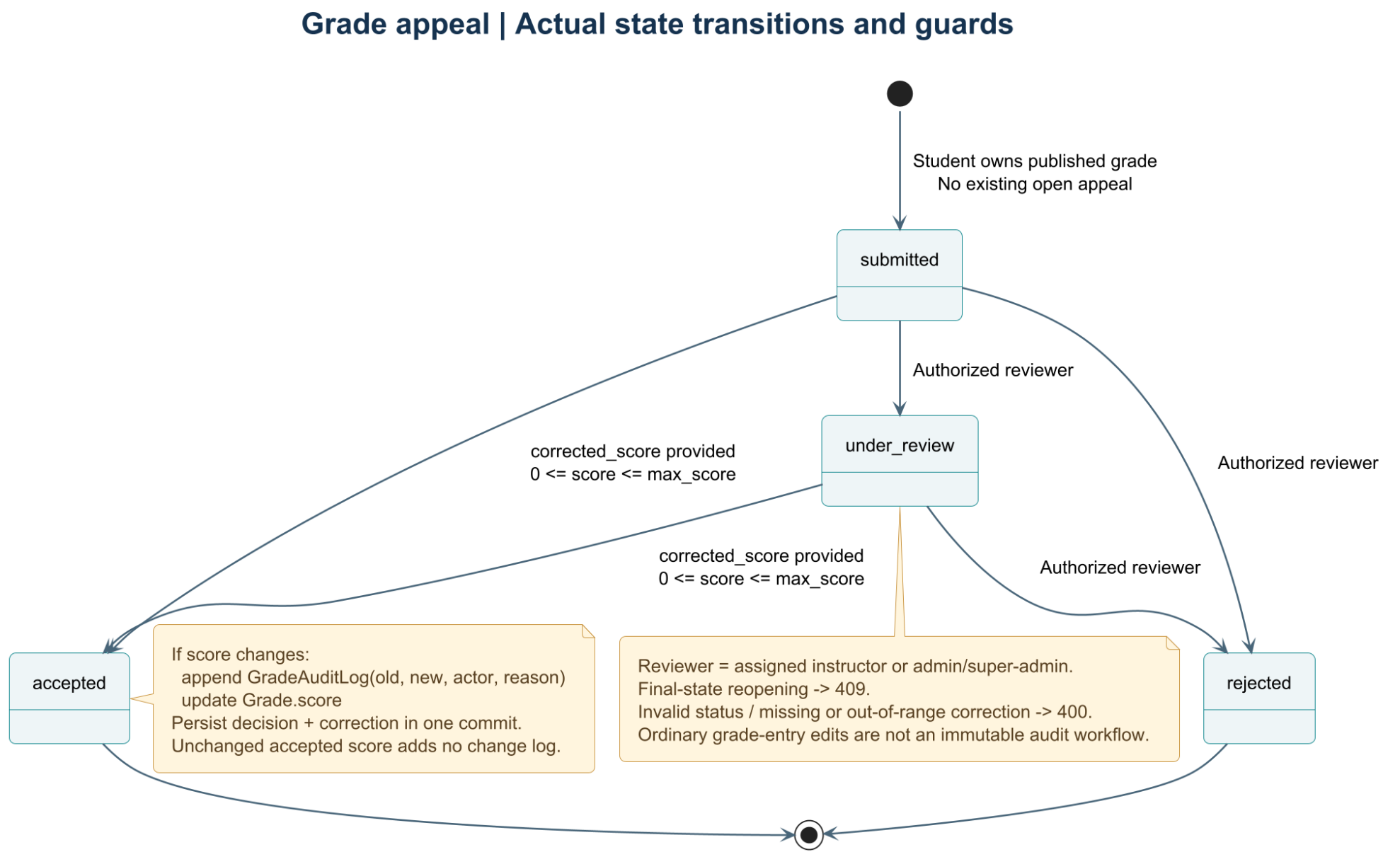


Figure 14: Grade appeal state machine. Only implemented transitions are shown; accepted corrections have explicit score and audit guards.

Editable source: `diagrams/13-appeal-states.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.15 Payroll generation and approval activity

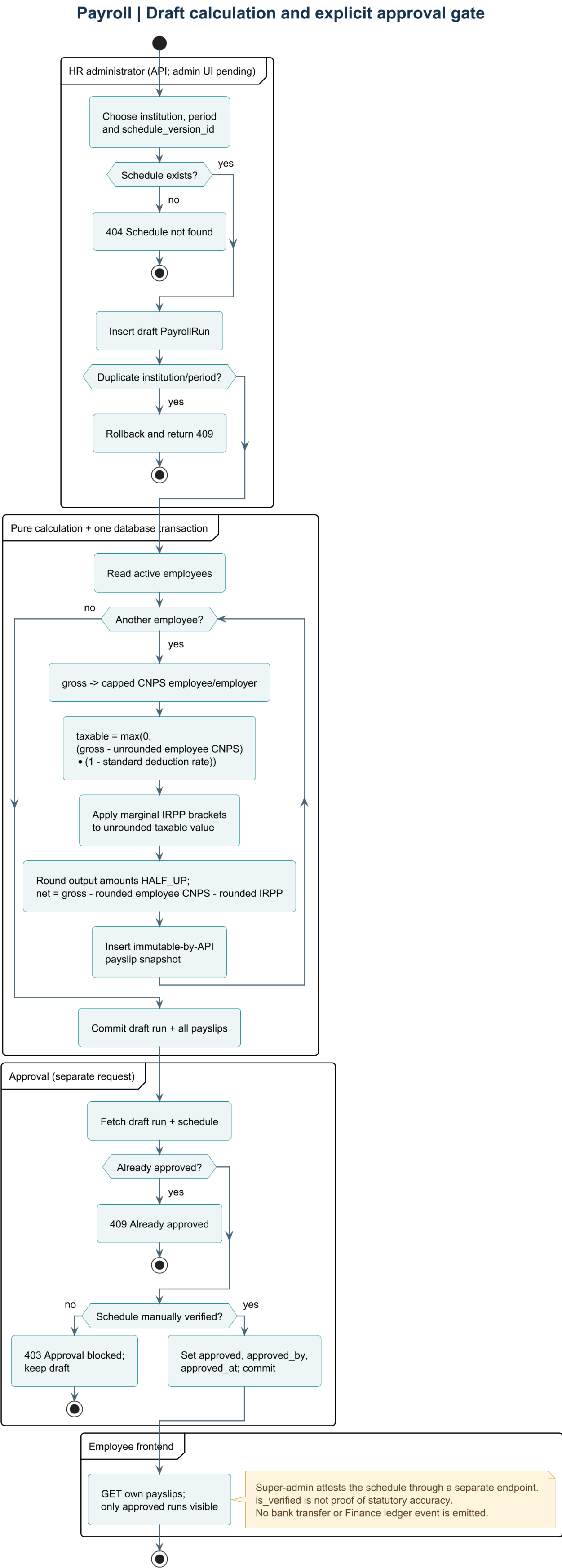


Figure 15: Payroll generation and approval activity. Pure calculation, snapshot creation and manual statutory verification are separate responsibilities.

Editable source: `diagrams/14-payroll-activity.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.16 Leave decision and notification sequence

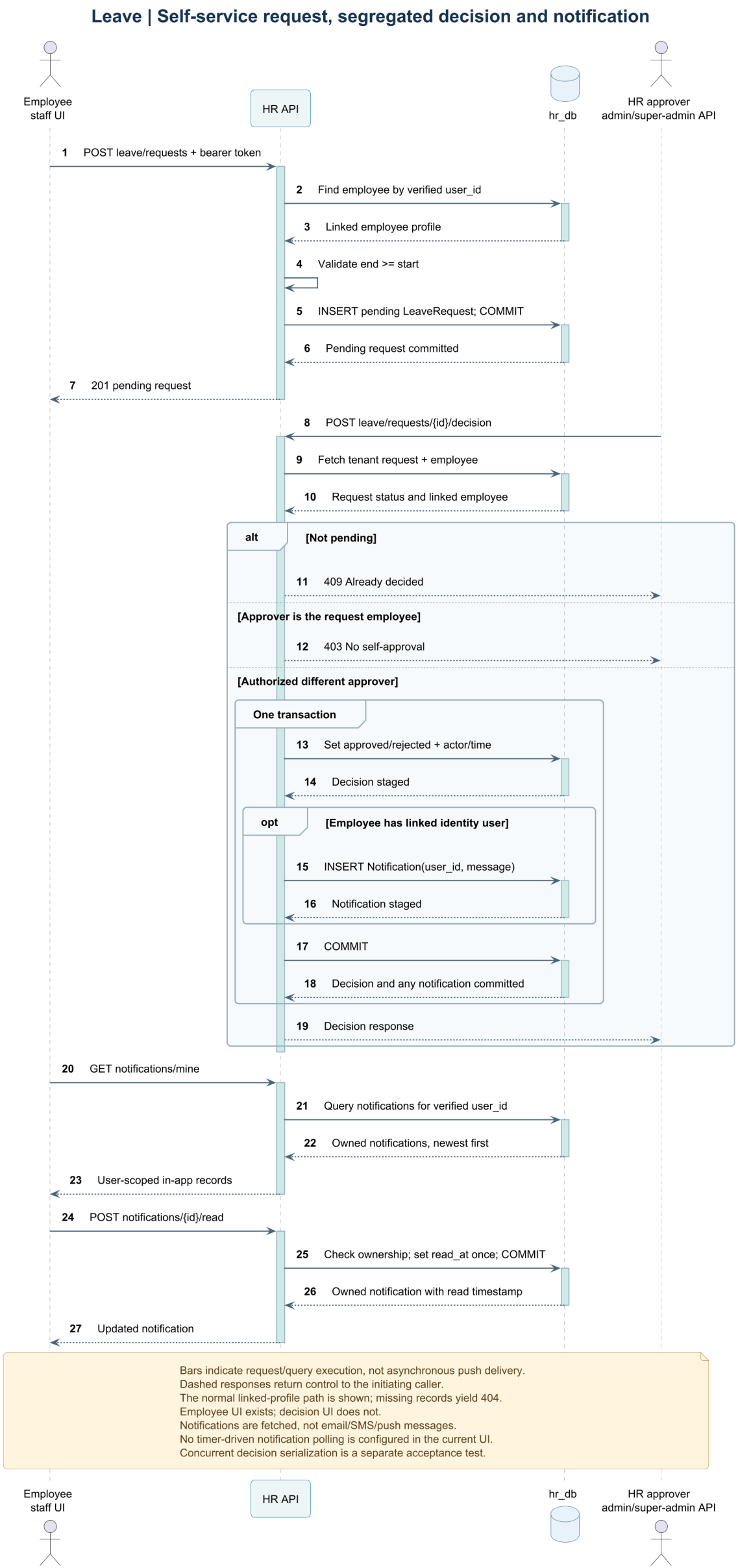


Figure 16: Leave decision and notification sequence. Decision segregation and in-app notification persistence are shown without fictitious email delivery.

Editable source: `diagrams/15-leave-sequence.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.17 Container deployment view

Deployment | Compose baseline and production intentions

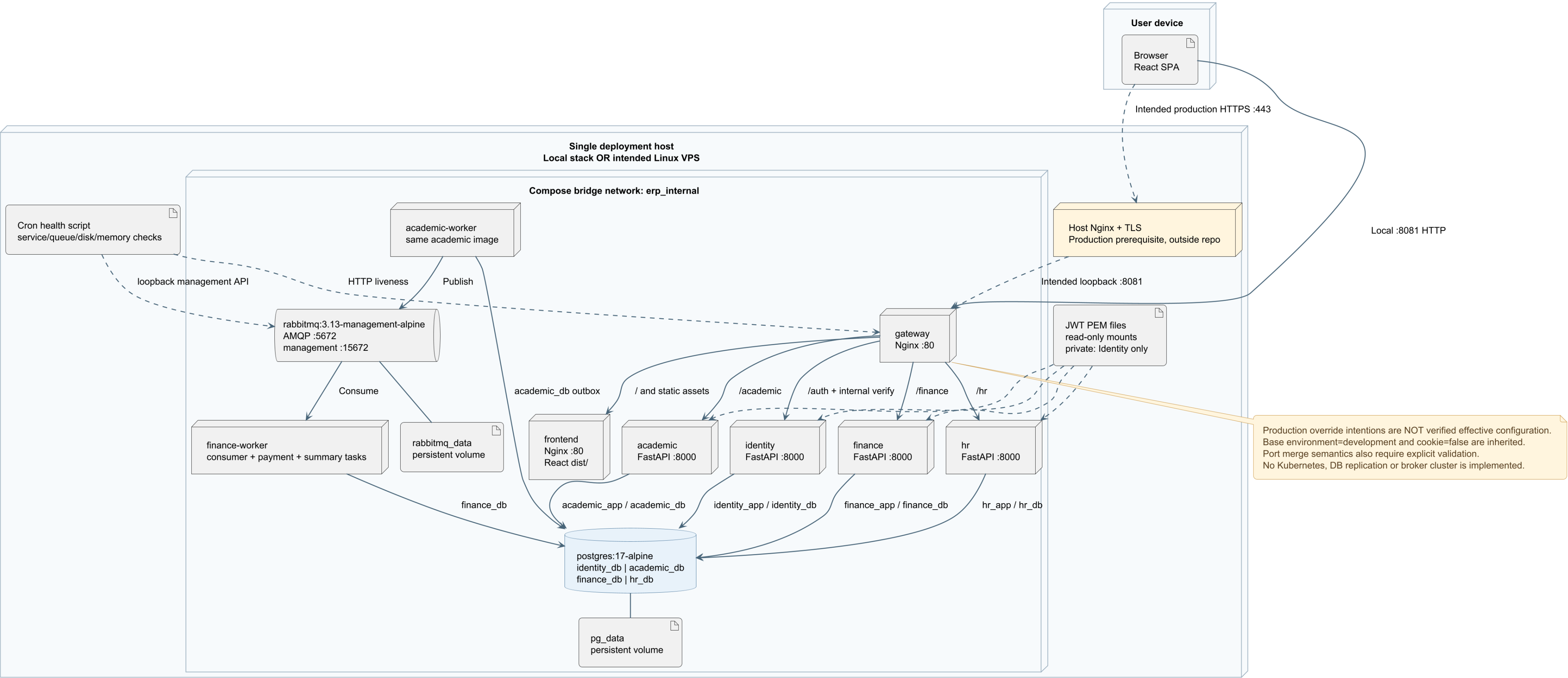


Figure 17: Container deployment view. All ten Compose containers, volumes and network paths are distinguished from unverified production intentions.

Editable source: `diagrams/16-deployment.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.18 Target full-system recovery activity

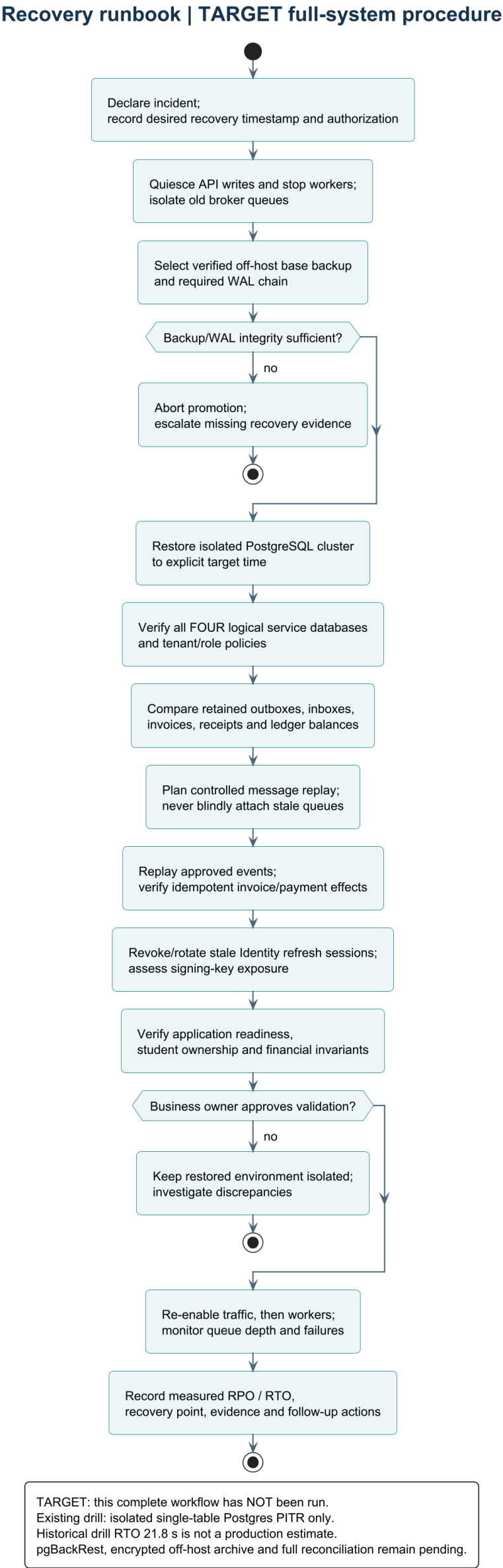


Figure 18: Target full-system recovery activity. This is a proposed operational procedure, not the already-run single-table PITR drill.

Editable source: `diagrams/17-recovery-activity.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.19 Conceptual scaling design

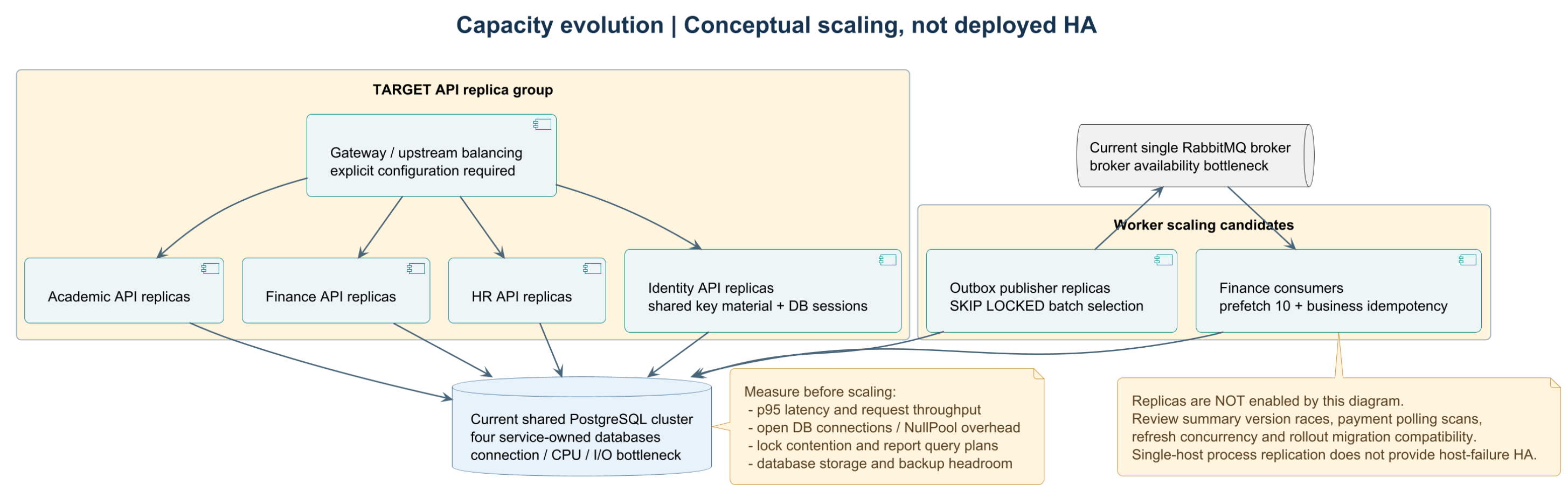


Figure 19: Conceptual scaling design. Replica groups are design candidates; shared infrastructure and concurrency limits remain explicit.

Editable source: `diagrams/18-scaling.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.20 Browser session state machine

Browser session | Observable state and retry policy

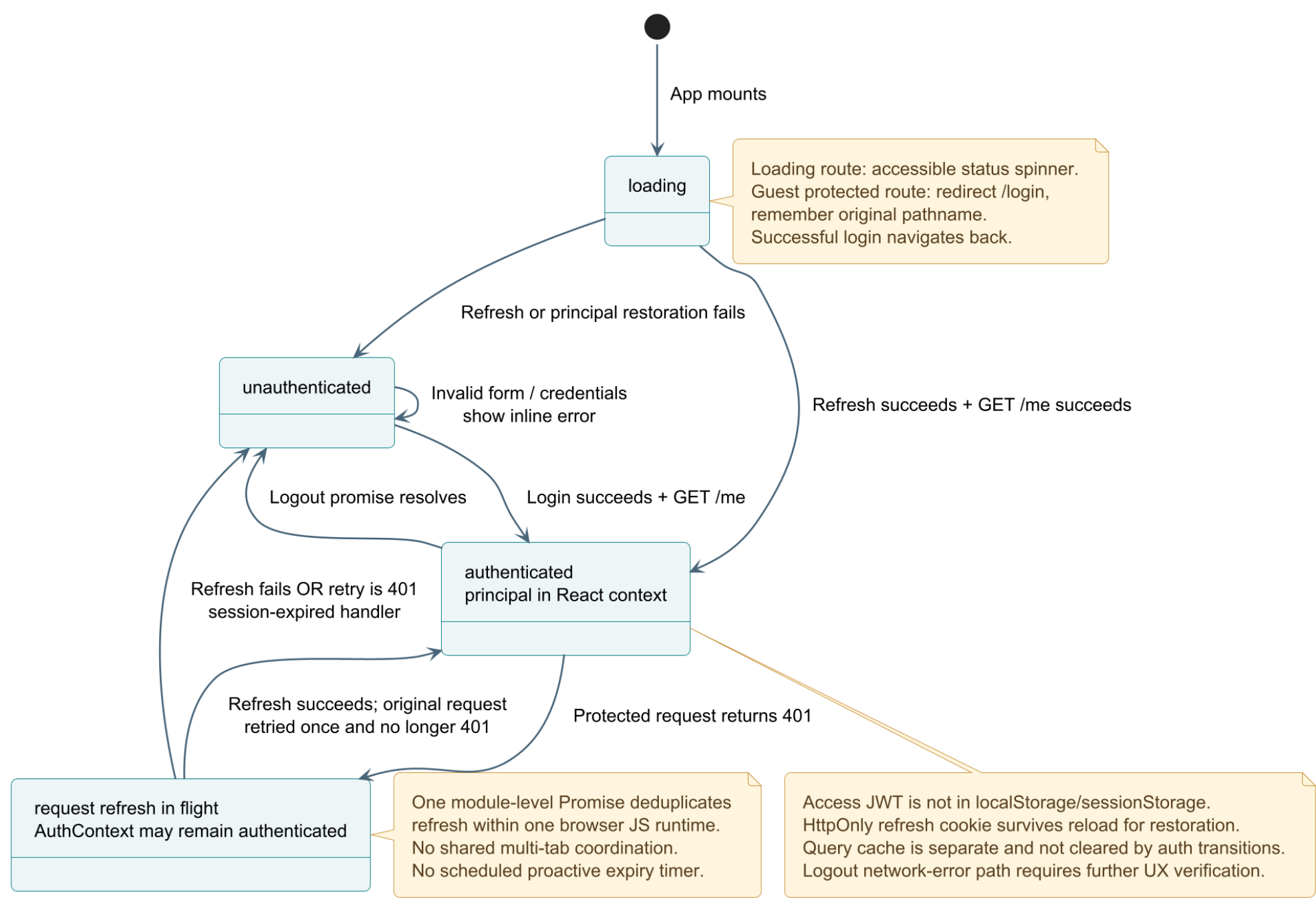


Figure 20: Browser session state machine. Startup restoration and reactive 401 handling are modeled without a nonexistent proactive expiry timer.

Editable source: `diagrams/19-browser-session.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.21 Exam scheduling conflict activity

Exam scheduling | Serialized conflict decision

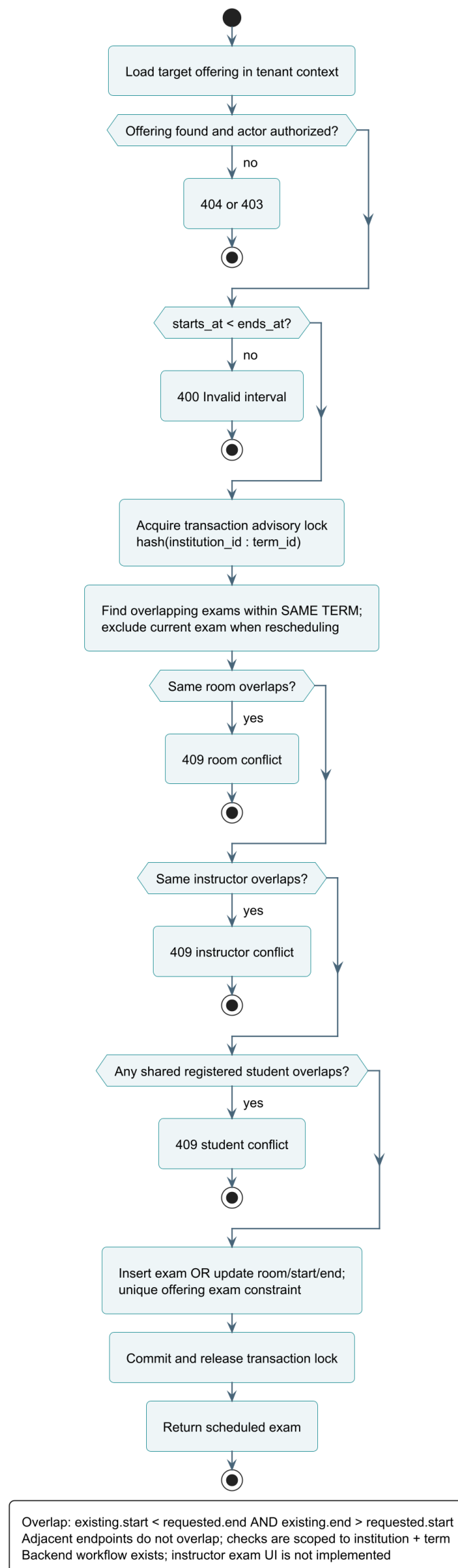


Figure 21: Exam scheduling conflict activity. The actual institution/term lock and interval conflict checks define the protected invariant.

Editable source: `diagrams/20-exam-activity.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

D.22 Aggregate stock movement activity

Asset movement | Aggregate stock invariant

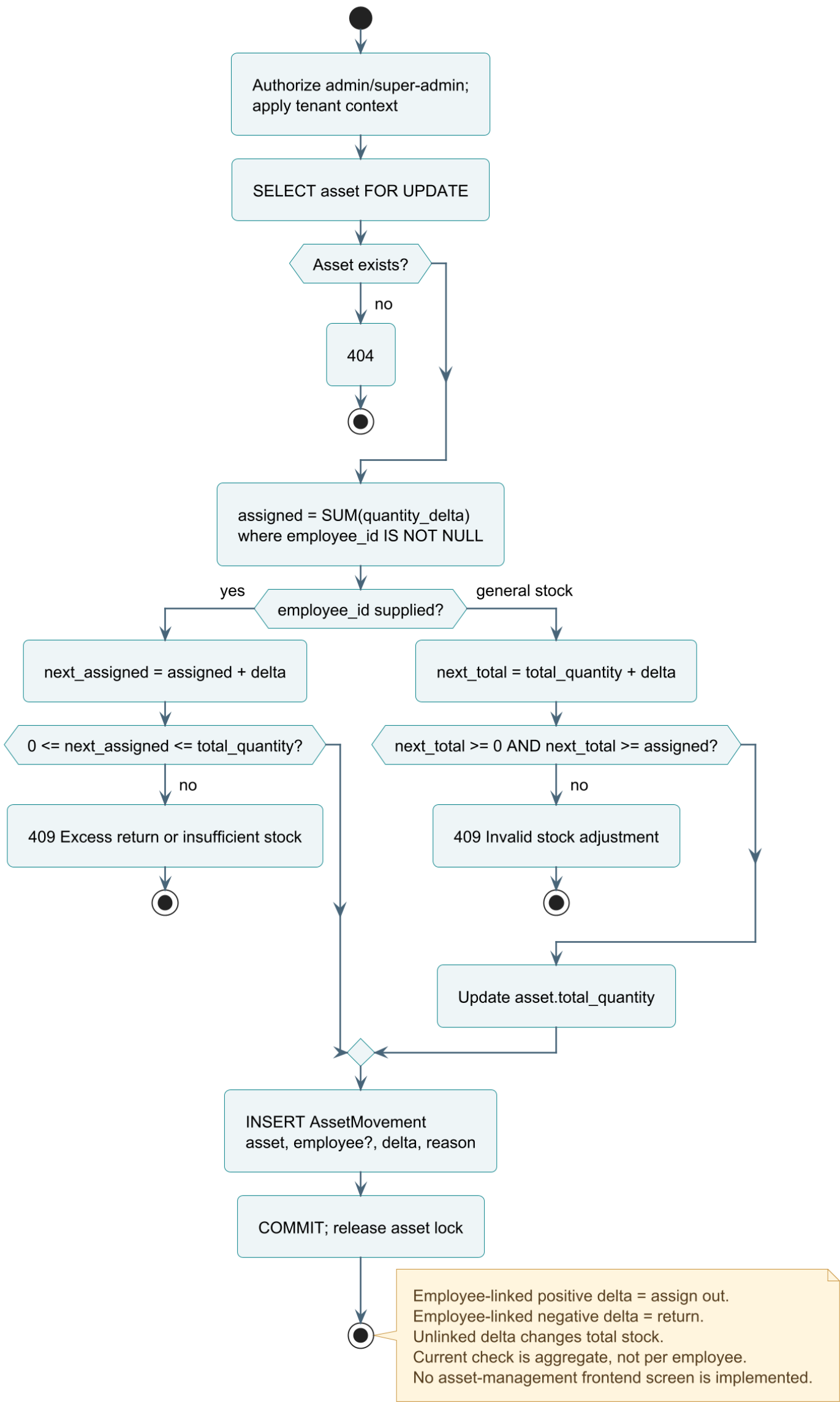


Figure 22: Aggregate stock movement activity. The flow protects aggregate on-hand quantity, not an unimplemented per-employee return constraint.

Editable source: `diagrams/21-stock-activity.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

E Generated service-local route catalogue

Generated by `inventory.py` from Python syntax trees without importing services, connecting to a database or reading secrets. It enumerates actual decorated handlers and router prefixes. Health and internal routes are service-local, not necessarily public gateway routes. Return annotations may name ORM classes where no explicit response model is declared. Access policy is explained in the SRS role matrix and the owning handler; the table is not a runtime OpenAPI export.

E.1 Identity route inventory

Method	Service-local route	Response / handler evidence
GET	/healthz	dict[str, str] services/identity/app/main.py:29
GET	/readyz	dict[str, str] services/identity/app/main.py:35
POST	/api/v1/auth/login	AccessTokenResponse services/identity/app/routers/auth.py:65
POST	/api/v1/auth/refresh	AccessTokenResponse services/identity/app/routers/auth.py:100
POST	/api/v1/auth/logout	None services/identity/app/routers/auth.py:163
GET	/api/v1/auth/me	MeResponse services/identity/app/routers/auth.py:180
GET	/internal/verify	dict services/identity/app/routers/internal.py:9

E.2 Academic route inventory

Method	Service-local route	Response / handler evidence
GET	/healthz	dict[str, str] services/academic/app/main.py:35
GET	/readyz	dict[str, str] services/academic/app/main.py:41
GET	/api/v1/academic/me	PrincipalResponse services/academic/app/main.py:50
GET	/api/v1/academic/programs	list[ProgramResponse] services/academic/app/routers/academic.py:15
GET	/api/v1/academic/terms	list[TermResponse] services/academic/app/routers/academic.py:21
POST	/api/v1/academic/enrollments	EnrollmentResponse services/academic/app/routers/academic.py:27
GET	/api/v1/academic/enrollments	list[EnrollmentResponse] services/academic/app/routers/academic.py:102
POST	/api/v1/academic/courses	CourseResponse services/academic/app/routers/courses.py:32
GET	/api/v1/academic/courses	list[CourseResponse] services/academic/app/routers/courses.py:58
POST	/api/v1/academic/courses/{course_id}/prerequisites	PrerequisiteResponse services/academic/app/routers/courses.py:103
GET	/api/v1/academic/courses/{course_id}/prerequisites	list[PrerequisiteResponse] services/academic/app/routers/courses.py:144
POST	/api/v1/academic/course-offerings	CourseOfferingResponse services/academic/app/routers/courses.py:157
GET	/api/v1/academic/course-offerings	list[CourseOfferingResponse] services/academic/app/routers/courses.py:193

Method	Service-local route	Response / handler evidence
POST	/api/v1/academic/course-registrations	CourseRegistrationResponse services/academic/app/routers/courses.py:246
GET	/api/v1/academic/course-registrations	list[CourseRegistrationResponse] services/academic/app/routers/courses.py:304
POST	/api/v1/academic/attendance-sessions	AttendanceSessionResponse services/academic/app/routers/attendance.py:31
POST	/api/v1/academic/attendance-sessions/{session_id}/records	list[AttendanceRecordResponse] services/academic/app/routers/attendance.py:60
GET	/api/v1/academic/attendance-sessions/{session_id}/records	list[AttendanceRecordResponse] services/academic/app/routers/attendance.py:98
GET	/api/v1/academic/attendance-sessions	list[AttendanceSessionResponse] services/academic/app/routers/attendance.py:111
POST	/api/v1/academic/assessments	AssessmentResponse services/academic/app/routers/grades.py:36
GET	/api/v1/academic/assessments	list[AssessmentResponse] services/academic/app/routers/grades.py:59
POST	/api/v1/academic/assessments/{assessment_id}/grades	list[GradeResponse] services/academic/app/routers/grades.py:82
POST	/api/v1/academic/assessments/{assessment_id}/publish	list[GradeResponse] services/academic/app/routers/grades.py:119
GET	/api/v1/academic/assessments/{assessment_id}/grades	list[GradeResponse] services/academic/app/routers/grades.py:136
POST	/api/v1/academic/grades/{grade_id}/appeals	GradeAppealResponse services/academic/app/routers/grades.py:151
GET	/api/v1/academic/grades/{grade_id}/appeals	list[GradeAppealResponse] services/academic/app/routers/grades.py:185
POST	/api/v1/academic/appeals/{appeal_id}/decision	GradeAppealResponse services/academic/app/routers/grades.py:207
POST	/api/v1/academic/exam-schedules	ExamScheduleResponse services/academic/app/routers/exams.py:103
PUT	/api/v1/academic/exam-schedules/{exam_schedule_id}	ExamScheduleResponse services/academic/app/routers/exams.py:156
GET	/api/v1/academic/exam-schedules	list[ExamScheduleResponse] services/academic/app/routers/exams.py:196
GET	/api/v1/academic/course-offerings/{course_offering_id}/at-risk	list[AtRiskStatus] services/academic/app/routers/reports.py:100
GET	/api/v1/academic/me/at-risk	list[AtRiskStatus] services/academic/app/routers/reports.py:123
GET	/api/v1/academic/reports/transcript	Response services/academic/app/routers/reports.py:166
GET	/api/v1/academic/reports/attendance-summary	Response services/academic/app/routers/reports.py:199

E.3 Finance route inventory

Method	Service-local route	Response / handler evidence
GET	/healthz	dict[str, str] services/finance/app/main.py:35
GET	/readyz	dict[str, str] services/finance/app/main.py:41
GET	/api/v1/finance/me	PrincipalResponse services/finance/app/main.py:50
POST	/api/v1/finance/fee-schedules	FeeScheduleResponse services/finance/app/routers/finance.py:21
GET	/api/v1/finance/invoices	list[InvoiceResponse] services/finance/app/routers/finance.py:48

Method	Service-local route	Response / handler evidence
GET	/api/v1/finance/invoices/{invoice_id}	InvoiceResponse services/finance/app/routers/finance.py:63
POST	/api/v1/finance/invoices/{invoice_id}/payment-intents	PaymentIntentResponse services/finance/app/routers/payments.py:32
GET	/api/v1/finance/payment-intents/{intent_id}	PaymentIntentResponse services/finance/app/routers/payments.py:80
POST	/api/v1/finance/payments/callback	dict services/finance/app/routers/payments.py:105
POST	/api/v1/finance/expenses	ExpenseResponse services/finance/app/routers/expenses.py:22
GET	/api/v1/finance/expenses	list[ExpenseResponse] services/finance/app/routers/expenses.py:52
GET	/api/v1/finance/ledger-entries	list[LedgerEntryResponse] services/finance/app/routers/expenses.py:61
POST	/api/v1/finance/campaigns	CampaignResponse services/finance/app/routers/campaigns.py:24
GET	/api/v1/finance/campaigns	list[CampaignResponse] services/finance/app/routers/campaigns.py:42
POST	/api/v1/finance/campaigns/{campaign_id}/leads	LeadResponse services/finance/app/routers/campaigns.py:51
GET	/api/v1/finance/campaigns/{campaign_id}/leads	list[LeadResponse] services/finance/app/routers/campaigns.py:70
POST	/api/v1/finance/leads/{lead_id}/convert	LeadResponse services/finance/app/routers/campaigns.py:80
GET	/api/v1/finance/campaigns/{campaign_id}/roi	CampaignRoiResponse services/finance/app/routers/campaigns.py:96
GET	/api/v1/finance/summaries	list[FinancialSummaryResponse] services/finance/app/routers/summaries.py:19
POST	/api/v1/finance/summaries/regenerate	FinancialSummaryResponse services/finance/app/routers/summaries.py:32
GET	/api/v1/finance/invoices/{invoice_id}/receipt	Response services/finance/app/routers/reports.py:20

E.4 Hr route inventory

Method	Service-local route	Response / handler evidence
GET	/healthz	dict[str, str] services/hr/app/main.py:35
GET	/readyz	dict[str, str] services/hr/app/main.py:41
GET	/api/v1/hr/me	PrincipalResponse services/hr/app/main.py:50
POST	/api/v1/hr/positions	PositionResponse services/hr/app/routers/recruitment.py:27
GET	/api/v1/hr/positions	list[PositionResponse] services/hr/app/routers/recruitment.py:44
POST	/api/v1/hr/candidates	CandidateResponse services/hr/app/routers/recruitment.py:53
GET	/api/v1/hr/candidates	list[CandidateResponse] services/hr/app/routers/recruitment.py:75
PATCH	/api/v1/hr/candidates/{candidate_id}/stage	CandidateResponse services/hr/app/routers/recruitment.py:84
POST	/api/v1/hr/candidates/{candidate_id}/hire	EmployeeResponse services/hr/app/routers/recruitment.py:102
POST	/api/v1/hr/attendance/shifts	ShiftResponse services/hr/app/routers/attendance.py:39

Method	Service-local route	Response / handler evidence
GET	/api/v1/hr/attendance/shifts	list[ShiftResponse] services/hr/app/routers/attendance.py:60
POST	/api/v1/hr/attendance/shifts/{shift_id}/qr-token	QRTokenResponse services/hr/app/routers/attendance.py:69
POST	/api/v1/hr/attendance/check-in	AttendanceRecordResponse services/hr/app/routers/attendance.py:91
GET	/api/v1/hr/attendance/mine	list[AttendanceRecordResponse] services/hr/app/routers/attendance.py:120
POST	/api/v1/hr/leave/requests	LeaveRequestResponse services/hr/app/routers/leave.py:33
GET	/api/v1/hr/leave/requests/mine	list[LeaveRequestResponse] services/hr/app/routers/leave.py:56
GET	/api/v1/hr/leave/requests	list[LeaveRequestResponse] services/hr/app/routers/leave.py:68
POST	/api/v1/hr/leave/requests/{request_id}/decision	LeaveRequestResponse services/hr/app/routers/leave.py:77
GET	/api/v1/hr/notifications/mine	list[NotificationResponse] services/hr/app/routers/leave.py:113
POST	/api/v1/hr/notifications/{notification_id}/read	NotificationResponse services/hr/app/routers/leave.py:126
POST	/api/v1/hr/performance/reviews	PerformanceReviewResponse services/hr/app/routers/performance.py:19
GET	/api/v1/hr/performance/reviews	list[PerformanceReviewResponse] services/hr/app/routers/performance.py:42
GET	/api/v1/hr/performance/reviews/mine	list[PerformanceReviewResponse] services/hr/app/routers/performance.py:51
POST	/api/v1/hr/assets	AssetResponse services/hr/app/routers/assets.py:17
GET	/api/v1/hr/assets	list[AssetResponse] services/hr/app/routers/assets.py:34
POST	/api/v1/hr/assets/movements	AssetMovementResponse services/hr/app/routers/assets.py:43
GET	/api/v1/hr/assets/{asset_id}/movements	list[AssetMovementResponse] services/hr/app/routers/assets.py:98
POST	/api/v1/hr/payroll/schedules	PayrollScheduleResponse services/hr/app/routers/payroll.py:30
GET	/api/v1/hr/payroll/schedules	list[PayrollScheduleResponse] services/hr/app/routers/payroll.py:56
PATCH	/api/v1/hr/payroll/schedules/{schedule_id}/verify	PayrollScheduleResponse services/hr/app/routers/payroll.py:67
POST	/api/v1/hr/payroll/runs	PayrollRunResponse services/hr/app/routers/payroll.py:88
POST	/api/v1/hr/payroll/runs/{run_id}/approve	PayrollRunResponse services/hr/app/routers/payroll.py:149
GET	/api/v1/hr/payroll/runs	list[PayrollRunResponse] services/hr/app/routers/payroll.py:180
GET	/api/v1/hr/payroll/runs/{run_id}/payslips	list[PayslipResponse] services/hr/app/routers/payroll.py:189
GET	/api/v1/hr/payroll/payslips/mine	list[PayslipResponse] services/hr/app/routers/payroll.py:199

F Generated persisted-model dictionary

The catalogue lists mapped-column attributes, Python ORM annotations and explicit column-level metadata. It intentionally excludes ORM relationship properties. An omitted `nullable` keyword does not prove nullability; consult SQLAlchemy annotation inference and the migration DDL. Composite/partial uniqueness, CHECKs, indexes and RLS policies must be read from model table arguments and migrations. The four ERDs provide navigable structure; this appendix provides field detail.

F.1 Identity persisted models

F.1.1 institutions (Institution)

Evidence: `services/identity/app/models/identity.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>name</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>slug</code>	<code>Mapped[str]</code>	<code>nullable=False; unique=True</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.1.2 campuses (Campus)

Evidence: `services/identity/app/models/identity.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False; FK institutions.id</code>
<code>name</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.1.3 users (User)

Evidence: `services/identity/app/models/identity.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID None]</code>	<code>nullable=True; FK institutions.id</code>
<code>campus_id</code>	<code>Mapped[uuid.UUID None]</code>	<code>nullable=True; FK campuses.id</code>
<code>email</code>	<code>Mapped[str]</code>	<code>nullable=False; unique=True</code>
<code>password_hash</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>full_name</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>role</code>	<code>Mapped[Role]</code>	<code>nullable=False</code>
<code>is_active</code>	<code>Mapped[bool]</code>	<code>nullable=False</code>
<code>failed_login_attempts</code>	<code>Mapped[int]</code>	<code>nullable=False</code>
<code>locked_until</code>	<code>Mapped[datetime None]</code>	<code>nullable=True</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration
<code>updated_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.1.4 refresh_sessions (RefreshSession)

Evidence: `services/identity/app/models/identity.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
user_id	Mapped[uuid.UUID]	nullable=False; FK users.id
institution_id	Mapped[uuid.UUID None]	nullable=True; FK institutions.id
family_id	Mapped[uuid.UUID]	nullable=False
token_hash	Mapped[str]	nullable=False; unique=True
issued_at	Mapped[datetime]	See model / migration
expires_at	Mapped[datetime]	nullable=False
revoked_at	Mapped[datetime None]	nullable=True

F.2 Academic persisted models

F.2.1 programs (Program)

Evidence: `services/academic/app/models/academic.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
name	Mapped[str]	nullable=False
code	Mapped[str]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.2 terms (Term)

Evidence: `services/academic/app/models/academic.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
name	Mapped[str]	nullable=False
starts_on	Mapped[datetime]	nullable=False
ends_on	Mapped[datetime]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.3 enrollments (Enrollment)

Evidence: `services/academic/app/models/academic.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
campus_id	Mapped[uuid.UUID]	nullable=False
student_id	Mapped[uuid.UUID]	nullable=False
program_id	Mapped[uuid.UUID]	nullable=False; FK programs.id
term_id	Mapped[uuid.UUID]	nullable=False; FK terms.id
status	Mapped[EnrollmentStatus]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.4 outbox_events (OutboxEvent)

Evidence: services/academic/app/models/academic.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
event_type	Mapped[str]	nullable=False
schema_version	Mapped[int]	nullable=False
correlation_id	Mapped[uuid.UUID]	nullable=False
payload	Mapped[dict]	nullable=False
created_at	Mapped[datetime]	See model / migration
published_at	Mapped[datetime None]	nullable=True

F.2.5 attendance_sessions (AttendanceSession)

Evidence: services/academic/app/models/attendance.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
course_offering_id	Mapped[uuid.UUID]	nullable=False; FK course_offerings.id
session_date	Mapped[date]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.6 attendance_records (AttendanceRecord)

Evidence: services/academic/app/models/attendance.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
attendance_session_id	Mapped[uuid.UUID]	nullable=False; FK attendance_sessions.id
student_id	Mapped[uuid.UUID]	nullable=False
present	Mapped[bool]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.7 courses (Course)

Evidence: services/academic/app/models/courses.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
program_id	Mapped[uuid.UUID]	nullable=False; FK programs.id
code	Mapped[str]	nullable=False
name	Mapped[str]	nullable=False
credits	Mapped[int]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.8 course_prerequisites (CoursePrerequisite)

Evidence: services/academic/app/models/courses.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
course_id	Mapped[uuid.UUID]	nullable=False; FK courses.id
prerequisite_course_id	Mapped[uuid.UUID]	nullable=False; FK courses.id
created_at	Mapped[datetime]	See model / migration

F.2.9 course_offerings (CourseOffering)

Evidence: services/academic/app/models/courses.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
course_id	Mapped[uuid.UUID]	nullable=False; FK courses.id
term_id	Mapped[uuid.UUID]	nullable=False; FK terms.id
instructor_id	Mapped[uuid.UUID]	nullable=False
room	Mapped[str]	nullable=False
capacity	Mapped[int]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.10 course_registrations (CourseRegistration)

Evidence: services/academic/app/models/courses.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
student_id	Mapped[uuid.UUID]	nullable=False
enrollment_id	Mapped[uuid.UUID]	nullable=False; FK enrollments.id
course_offering_id	Mapped[uuid.UUID]	nullable=False; FK course_offerings.id
created_at	Mapped[datetime]	See model / migration

F.2.11 exam_schedules (ExamSchedule)

Evidence: services/academic/app/models/exams.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
term_id	Mapped[uuid.UUID]	nullable=False; FK terms.id
course_offering_id	Mapped[uuid.UUID]	nullable=False; FK course_offerings.id
room	Mapped[str]	nullable=False
starts_at	Mapped[datetime]	nullable=False
ends_at	Mapped[datetime]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.12 assessments (Assessment)

Evidence: services/academic/app/models/grades.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
course_offering_id	Mapped[uuid.UUID]	nullable=False; FK course_offerings.id
name	Mapped[str]	nullable=False
max_score	Mapped[int]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.13 grades (Grade)

Evidence: services/academic/app/models/grades.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
assessment_id	Mapped[uuid.UUID]	nullable=False; FK assessments.id
student_id	Mapped[uuid.UUID]	nullable=False
score	Mapped[Decimal]	nullable=False
published	Mapped[bool]	nullable=False
created_at	Mapped[datetime]	See model / migration
updated_at	Mapped[datetime]	See model / migration

F.2.14 grade_audit_logs (GradeAuditLog)

Evidence: services/academic/app/models/grades.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
grade_id	Mapped[uuid.UUID]	nullable=False; FK grades.id
changed_by	Mapped[uuid.UUID]	nullable=False
old_score	Mapped[Decimal]	nullable=False
new_score	Mapped[Decimal]	nullable=False
reason	Mapped[str]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.2.15 grade_appeals (GradeAppeal)

Evidence: services/academic/app/models/grades.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
grade_id	Mapped[uuid.UUID]	nullable=False; FK grades.id
student_id	Mapped[uuid.UUID]	nullable=False
reason	Mapped[str]	nullable=False
status	Mapped[AppealStatus]	nullable=False
reviewer_notes	Mapped[str None]	nullable=True
decided_by	Mapped[uuid.UUID None]	nullable=True

Attribute	Python ORM annotation	Explicit column metadata
<code>decided_at</code>	<code>Mapped[datetime None]</code>	<code>nullable=True</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.3 Finance persisted models

F.3.1 `fee_schedules` (`FeeSchedule`)

Evidence: `services/finance/app/models/finance.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>program_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>term_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>amount_xaf</code>	<code>Mapped[int]</code>	<code>nullable=False</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.3.2 `invoices` (`Invoice`)

Evidence: `services/finance/app/models/finance.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>enrollment_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>student_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>amount_xaf</code>	<code>Mapped[int]</code>	<code>nullable=False</code>
<code>status</code>	<code>Mapped[InvoiceStatus]</code>	<code>nullable=False</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.3.3 `inbox_events` (`InboxEvent`)

Evidence: `services/finance/app/models/finance.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>event_id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>event_type</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>processed_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.3.4 `ledger_entries` (`LedgerEntry`)

Evidence: `services/finance/app/models/ledger.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>posting_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>entry_type</code>	<code>Mapped[LedgerEntryType]</code>	<code>nullable=False</code>
<code>direction</code>	<code>Mapped[LedgerDirection]</code>	<code>nullable=False</code>

Attribute	Python ORM annotation	Explicit column metadata
amount_xaf	Mapped[int]	nullable=False
reference_type	Mapped[str]	nullable=False
reference_id	Mapped[uuid.UUID]	nullable=False
description	Mapped[str]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.3.5 receipts (Receipt)

Evidence: `services/finance/app/models/ledger.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
invoice_id	Mapped[uuid.UUID]	nullable=False
payment_intent_id	Mapped[uuid.UUID]	nullable=False
amount_xaf	Mapped[int]	nullable=False
issued_at	Mapped[datetime]	See model / migration

F.3.6 expenses (Expense)

Evidence: `services/finance/app/models/ledger.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
category	Mapped[str]	nullable=False
amount_xaf	Mapped[int]	nullable=False
description	Mapped[str]	nullable=False
recorded_by	Mapped[uuid.UUID]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.3.7 campaigns (Campaign)

Evidence: `services/finance/app/models/marketing.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
name	Mapped[str]	nullable=False
cost_xaf	Mapped[int]	nullable=False
starts_on	Mapped[date]	nullable=False
ends_on	Mapped[date]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.3.8 leads (Lead)

Evidence: `services/finance/app/models/marketing.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
campaign_id	Mapped[uuid.UUID]	nullable=False; FK campaigns.id
status	Mapped[LeadStatus]	nullable=False
student_id	Mapped[uuid.UUID None]	nullable=True
created_at	Mapped[datetime]	See model / migration

F.3.9 payment_intents (PaymentIntent)

Evidence: `services/finance/app/models/payments.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
invoice_id	Mapped[uuid.UUID]	nullable=False; FK invoices.id
amount_xaf	Mapped[int]	nullable=False
provider	Mapped[str]	nullable=False
provider_reference	Mapped[str]	nullable=False; unique=True
payer_msisdn	Mapped[str]	nullable=False
status	Mapped[PaymentIntentStatus]	nullable=False
created_at	Mapped[datetime]	See model / migration
updated_at	Mapped[datetime]	See model / migration

F.3.10 financial_summaries (FinancialSummary)

Evidence: `services/finance/app/models/summaries.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
period	Mapped[date]	nullable=False
version	Mapped[int]	nullable=False
total_revenue_xaf	Mapped[int]	nullable=False
total_expenses_xaf	Mapped[int]	nullable=False
net_xaf	Mapped[int]	nullable=False
generated_at	Mapped[datetime]	See model / migration

F.4 Hr persisted models

F.4.1 assets (Asset)

Evidence: `services/hr/app/models/assets.py`

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
name	Mapped[str]	nullable=False
category	Mapped[str]	nullable=False
total_quantity	Mapped[int]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.2 asset_movements (AssetMovement)

Evidence: services/hr/app/models/assets.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
asset_id	Mapped[uuid.UUID]	nullable=False; FK assets.id
employee_id	Mapped[uuid.UUID None]	nullable=True
quantity_delta	Mapped[int]	nullable=False
reason	Mapped[str]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.3 attendance_shifts (AttendanceShift)

Evidence: services/hr/app/models/attendance.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
employee_id	Mapped[uuid.UUID]	nullable=False; FK employees.id
starts_at	Mapped[datetime]	nullable=False
ends_at	Mapped[datetime]	nullable=False
qr_token_jti	Mapped[uuid.UUID None]	nullable=True; unique=True
created_at	Mapped[datetime]	See model / migration

F.4.4 attendance_records (AttendanceRecord)

Evidence: services/hr/app/models/attendance.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
shift_id	Mapped[uuid.UUID]	nullable=False; FK attendance_shifts.id
employee_id	Mapped[uuid.UUID]	nullable=False
checked_in_at	Mapped[datetime]	See model / migration

F.4.5 employees (Employee)

Evidence: services/hr/app/models/employees.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
user_id	Mapped[uuid.UUID None]	nullable=True
candidate_id	Mapped[uuid.UUID None]	nullable=True; FK candidates.id
full_name	Mapped[str]	nullable=False
email	Mapped[str]	nullable=False
department	Mapped[str]	nullable=False
hire_date	Mapped[date]	nullable=False
gross_monthly_salary_xaf	Mapped[int]	nullable=False

Attribute	Python ORM annotation	Explicit column metadata
status	Mapped[EmployeeStatus]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.6 leave_requests (LeaveRequest)

Evidence: services/hr/app/models/leave.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
employee_id	Mapped[uuid.UUID]	nullable=False; FK employees.id
starts_on	Mapped[date]	nullable=False
ends_on	Mapped[date]	nullable=False
reason	Mapped[str]	nullable=False
status	Mapped[LeaveStatus]	nullable=False
decided_by	Mapped[uuid.UUID None]	nullable=True
decided_at	Mapped[datetime None]	nullable=True
created_at	Mapped[datetime]	See model / migration

F.4.7 notifications (Notification)

Evidence: services/hr/app/models/leave.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
user_id	Mapped[uuid.UUID]	nullable=False
message	Mapped[str]	nullable=False
notified_via_email	Mapped[bool]	nullable=False
created_at	Mapped[datetime]	See model / migration
read_at	Mapped[datetime None]	nullable=True

F.4.8 payroll_schedule_versions (PayrollScheduleVersion)

Evidence: services/hr/app/models/payroll.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
effective_from	Mapped[date]	nullable=False
is_verified	Mapped[bool]	nullable=False
cnps_employee_rate	Mapped[Decimal]	nullable=False
cnps_employer_rate	Mapped[Decimal]	nullable=False
cnps_ceiling_xaf	Mapped[int]	nullable=False
standard_deduction_rate	Mapped[Decimal]	nullable=False
irpp_brackets	Mapped[list]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.9 payroll_runs (PayrollRun)

Evidence: services/hr/app/models/payroll.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
period	Mapped[date]	nullable=False
schedule_version_id	Mapped[uuid.UUID]	nullable=False; FK payroll_schedule_versions.id
status	Mapped[PayrollRunStatus]	nullable=False
approved_by	Mapped[uuid.UUID None]	nullable=True
approved_at	Mapped[datetime None]	nullable=True
created_at	Mapped[datetime]	See model / migration

F.4.10 payslips (Payslip)

Evidence: services/hr/app/models/payroll.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
run_id	Mapped[uuid.UUID]	nullable=False; FK payroll_runs.id
employee_id	Mapped[uuid.UUID]	nullable=False
period	Mapped[date]	nullable=False
gross_xaf	Mapped[int]	nullable=False
cnps_employee_xaf	Mapped[int]	nullable=False
cnps_employer_xaf	Mapped[int]	nullable=False
taxable_base_xaf	Mapped[int]	nullable=False
irpp_xaf	Mapped[int]	nullable=False
net_xaf	Mapped[int]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.11 performance_reviews (PerformanceReview)

Evidence: services/hr/app/models/performance.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True
institution_id	Mapped[uuid.UUID]	nullable=False
employee_id	Mapped[uuid.UUID]	nullable=False; FK employees.id
reviewer_id	Mapped[uuid.UUID]	nullable=False
period	Mapped[date]	nullable=False
rating	Mapped[int]	nullable=False
comments	Mapped[str]	nullable=False
created_at	Mapped[datetime]	See model / migration

F.4.12 positions (Position)

Evidence: services/hr/app/models/recruitment.py

Attribute	Python ORM annotation	Explicit column metadata
id	Mapped[uuid.UUID]	primary_key=True

Attribute	Python ORM annotation	Explicit column metadata
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>title</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>department</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>status</code>	<code>Mapped[PositionStatus]</code>	<code>nullable=False</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

F.4.13 candidates (Candidate)

Evidence: `services/hr/app/models/recruitment.py`

Attribute	Python ORM annotation	Explicit column metadata
<code>id</code>	<code>Mapped[uuid.UUID]</code>	<code>primary_key=True</code>
<code>institution_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code>
<code>position_id</code>	<code>Mapped[uuid.UUID]</code>	<code>nullable=False</code> ; FK <code>positions.id</code>
<code>full_name</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>email</code>	<code>Mapped[str]</code>	<code>nullable=False</code>
<code>stage</code>	<code>Mapped[CandidateStage]</code>	<code>nullable=False</code>
<code>created_at</code>	<code>Mapped[datetime]</code>	See model / migration

G Setup, validation and operational handover

G.1 Clean local setup sequence

Prerequisites: Docker with Compose v2 supporting the override tags used by this repository; sibling frontend checkout; Python 3.13 for service checks; Node 22 for frontend local builds. Choose an isolated local environment and synthetic records. The following commands are an operator guide, not commands executed by the document build.

```
# From ERP_System_backend; preserve an existing .env.
Copy-Item .env.example .env
Set-Location services\identity
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe ..\..\scripts\generate_dev_jwt_keys.py
Set-Location ..\..
docker compose up -d --build
docker compose exec identity python -m alembic upgrade head
docker compose exec academic python -m alembic upgrade head
docker compose exec finance python -m alembic upgrade head
docker compose exec hr python -m alembic upgrade head
docker compose exec identity python -m scripts.seed
docker compose exec academic python -m scripts.seed
docker compose exec finance python -m scripts.seed
docker compose exec hr python -m scripts.seed
```

Do not overwrite an existing environment file without reviewing it. Use the matching interpreter for your platform, rather than blindly copying a machine-specific executable. Seed outputs are development credentials and fixtures, not real accounts. Verify seed identifier alignment, fee schedules and employee/user linkage before demonstrating dependent workflows. Never run seed scripts against production records. Start with gateway <http://localhost:8081>; inspect each API's **readyz**, not only gateway liveness. Workers have no Compose health check; verify event transfer by enrolling a synthetic student and observing a Finance invoice.

G.2 Existing test/build entry points

Scope	Existing commands / conditions
Backend service	In each service: ruffcheck. , mypyapp , pytest--cov=app--cov-fail-under=70 ; install that service's dev requirements and provide migrated PostgreSQL/key configuration as its fixtures require.
Contracts	In contracts : install declared requirements; run pytest .
Isolation	Run tests/integration/test_db_isolation.py against the isolated configured test cluster.
Frontend	npmci , npmrunlint , npmruntypecheck , npmruntest:coverage , npmrunbuild .
Frontend E2E	npmruntest:e2e ; requires live gateway, dedicated seeded credentials and browser installation. Reusing a rotating cookie across independent contexts is invalid.
Documentation	Run docs/specifications/build.ps1 with local Java/PlantUML, Python and Tectonic paths; see package README.

Documentation validation does not mutate runtime data or substitute for these application tests. Test evidence must identify both frontend and backend revisions.

G.3 Operational handover gates

1. Confirm domain, certificate, effective environment, cookie flags, listener binding and unique secrets before opening a production origin.
2. Provision an image publication process; record frontend/backend refs and actual image digests. Validate Jenkins syntax and credentials against the real controller.
3. Establish off-host backup, WAL continuity, retention and monitoring; test a representative full-cluster restore and event reconciliation.
4. Validate provider mode and payroll rate sources; keep simulations explicit.
5. Resolve mandatory UI/API gaps and collect real coverage, EXPLAIN/load and issue-board evidence.
6. Review legacy comments and historical reports for drift; source and verified effective runtime configuration

remain the authoritative implementation evidence.

G.4 Documentation verification contract

The build regenerates route/model tables, renders PlantUML locally to PNG/SVG and compiles both PDFs with Tectonic. It fails on diagram/compiler errors, undefined references or overfull boxes. PDF pagination, extractable text and rendered diagram legibility are checked separately during preparation. No source or diagram text is submitted to a public PlantUML server; downloaded tools/typesetting packages do not require sending repository content.

G.5 References and disclosure

Primary stakeholder source is the supplied SEN4121 Summer 2026 brief. Implementation references are backend `6e10c54`, frontend `6cc6859`, and the exact source paths in the SRS and generated appendices. Historical measurements come only from `docs/performance.md` and `docs/backup-recovery.md`. The diagrams and narrative are AI-assisted original documentation of this project, not copied standard text. Team review, signature, individual contribution statements and disclosure at defense are still required.